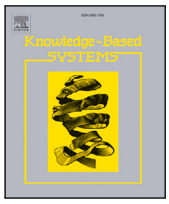
Knowledge-Based Systems 348 (2026) 116324



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





LGMT: Logic-Grounded Metamorphic Testing for evaluating the reasoning reliability of LLMs

Zenghui Zhou, Man Li, Xiaoke Fang, Xinyi Zhou, Weibin Lin, Zheng Zheng *

School of Automation Science and Electrical Engineering, Beihang University, Xueyuan Road, Haidian District, Beijing, 100191, China

ARTICLE INFO

Dataset link: https://anonymous.4open.science/r/LGMT_code-8A0D/README.md

Keywords:
Large Language Models
Metamorphic testing
Logical reasoning
First-order logic

ABSTRACT

Large Language Models (LLMs) achieve strong performance on logical reasoning benchmarks, yet their reliability remains uncertain. Existing evaluations rely on static benchmarks, which fail to assess robustness under logically equivalent transformations and often overestimate reasoning capability. We propose LGMT (Logic-Grounded Metamorphic Testing), an oracle-free framework that leverages first-order logic (FOL) to evaluate LLM reasoning. By deriving metamorphic relations from formal logical equivalences, LGMT constructs semantically invariant test cases and detects reasoning defects through cross-case consistency checking. Experiments on six state-of-the-art LLMs show that LGMT exposes substantial hidden defects missed by traditional reference-based evaluations. We further find that models are particularly sensitive to symbol-level and conclusion-level variations, and that advanced prompting such as Few-shot CoT only partially mitigates these issues. These results suggest that LLM evaluation should move beyond isolated correctness toward robustness under logical invariance. LGMT provides a principled and scalable approach for diagnosing reasoning failures.

1. Introduction

Large Language Models (LLMs) have been increasingly integrated into modern software systems, powering a wide range of applications such as automated question answering [1–4], code generation [5–7], and intelligent assistants [8,9]. In many of these applications, LLMs are expected not only to generate fluent responses, but also to perform reliable logical reasoning over given facts, rules, and constraints [10–12]. This property is critical for LLM-integrated software systems, because inconsistent reasoning under semantically equivalent inputs may lead to unreliable decisions even when the model appears accurate on individual benchmark instances [13–16]. As LLMs increasingly act as knowledge-intensive components in software systems, rigorously testing whether their reasoning results remain stable under equivalent semantics is a prerequisite for ensuring reliability before practical deployment [17–20].

However, testing the logical reasoning reliability of LLMs faces fundamental challenges. Currently, the dominant testing paradigm relies heavily on reference-based evaluation using static benchmarks (e.g., FOLIO [21], LogicBench [22], and LogiQA [23]). This paradigm inherently suffers from a severe *Test Oracle Problem*: manually labeling expected logical outputs for diverse inputs is prohibitively expensive and fundamentally cannot scale to support extensive, automated testing. Moreover, the static nature of these datasets makes them highly susceptible to *data contamination* (i.e., benchmark leakage) [24], where LLMs may inadvertently memorize the test cases during pre-training. This may create a false illusion of reasoning reliability rather than reflecting their true out-of-distribution generalization capabilities. Consequently, static benchmarks often suffer from poor timeliness and truncated lifecycles. To accurately diagnose the true capabilities of LLMs, evaluators are forced into an exhausting cycle of continuously constructing entirely new, unseen datasets [25]. This perpetual "cat-and-mouse" game between benchmarks and model training incurs an unsustainable maintenance cost, further exacerbating the oracle problem [26]. Furthermore, these static benchmarks typically formulate logical reasoning as binary, ternary, or multiple-choice classification tasks. Relying solely on such static labels for validation is intrinsically unreliable. Due to the restricted output space, LLMs can achieve seemingly high accuracy through random guessing or by exploiting spurious statistical artifacts (i.e., textual shortcuts) rather than engaging in genuine step-by-step logical deduction [27]. This *coincidental correctness* severely masks their underlying logical deficits [28].

To address these limitations, recent studies have introduced Metamorphic Testing (MT) as a zero-resource, black-box testing methodology for LLMs [28–31]. By examining whether the outputs of source and follow-up test cases satisfy predefined Metamorphic Relations (MRs), MT effectively alleviates the oracle problem [32]. Crucially, MT shifts

* Corresponding author.
E-mail address: zhengz@buaa.edu.cn (Z. Zheng).

https://doi.org/10.1016/j.knosys.2026.116324
Received 29 April 2026; Received in revised form 19 May 2026; Accepted 24 May 2026
Available online 8 June 2026
0950-7051/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

the evaluation focus from individual test case to the preservation of invariant properties. This characteristic makes MT inherently more resilient to data contamination, as simply memorizing training samples is insufficient to guarantee consistency across diverse MRs. Furthermore, the cross-case validation mechanism in MT helps mitigate coincidental correctness, as it is significantly more challenging for LLMs to maintain consistency through random guessing than to pass a single static test.

Despite its promise, current MT approaches for NLP tasks predominantly rely on *lexical-level* natural language perturbations, such as synonym substitution or syntactic paraphrasing [30,31,33,34]. Since natural language is inherently ambiguous, these informal input transformations frequently introduce *semantic drift* or grammatical inconsistencies. Recent large-scale empirical studies demonstrate that such natural-language-based MRs often yield high false positive rates [34]. Consequently, when a test fails, developers struggle to distinguish whether the failure stems from a genuine logical defect in the LLM or a follow-up test case that inadvertently altered the original underlying logic.

To overcome these limitations, we propose **LGMT**, a logic-grounded metamorphic testing framework for systematically evaluating the logical reasoning reliability of LLMs. The key novelty of LGMT lies in grounding metamorphic testing in First-Order Logic (FOL): instead of relying on informal lexical mutations, we derive MRs directly from formally verified FOL equivalence laws, such as De Morgan's laws, contraposition, and quantifier transformations. This design elevates metamorphic testing from lexical-level perturbation to *logic-preserving transformation*, providing a mechanism for generating follow-up test cases whose symbolic semantics are guaranteed by construction. As a result, LGMT substantially reduces the semantic drift that has long limited natural-language-based MT. More importantly, by validating whether LLMs preserve their judgments across logically equivalent test cases, LGMT moves evaluation beyond isolated correctness on static benchmarks and toward robustness under logical invariance. This allows the framework not only to expose hidden reasoning defects that traditional testing misses, but also to provide a formally grounded alternative for oracle-free reliability assessment of LLM reasoning.

To the best of our knowledge, this is among the first studies to ground MRs in FOL for systematically assessing the logical reasoning reliability of LLMs. The main contributions of this paper are summarized as follows:

- We propose **LGMT**, a metamorphic testing framework that derives input transformations from FOL and evaluates whether LLM entailment judgments remain consistent across source and follow-up cases.
- We design 20 types of FOL-grounded metamorphic relations across formula, symbol, premise, and conclusion-level transformations, enabling systematic generation of logic-preserving follow-up test cases.
- We provide a formal analysis of MR-E1 with respect to PNNF-oriented normalization, showing that the core equivalence rules are sufficient for reaching a canonical normalized representation under the proposed rewrite procedure.
- We construct 76,298 candidate metamorphic groups from three reasoning benchmarks and evaluate 3091 sampled groups across six LLMs and four prompting strategies.
- We empirically show that LLMs exhibit non-trivial inconsistency under FOL-grounded transformations, with particularly high sensitivity to symbol and conclusion-level variants, and an audit of sampled reports suggests a low false-report rate.

2. Preliminaries

In this section, we first introduce first-order logic and logical entailment, then formalize MT for LLM reasoning evaluation. We then present motivating examples to illustrate the limitations of current LLMs and to clarify why FOL-grounded MRs provide a more reliable alternative to lexical MRs.

2.1. First-order logic and logical entailment

Syntax of First-Order Logic. FOL is a formal symbolic system extensively utilized to represent complex facts and rules [35,36]. Let $\mathcal{L}$ be a first-order language defined over predicates (representing properties and relations), constants (denoting specific entities), and variables. The formulas in $\mathcal{L}$ are recursively constructed from these components using a set of logical connectives, including negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), implication ($\rightarrow$), and biconditional ($\leftrightarrow$) [21]. To enable quantification over entities within a domain, FOL further introduces universal ($\forall$) and existential ($\exists$) quantifiers. This compositional structure allows FOL to precisely capture the latent logic of natural language sentences, thereby translating linguistic forms into rigorous symbolic representations [15,37].

Formalizing Logical Reasoning. In the context of reasoning evaluation, a test case x is formally abstracted as a premise–conclusion pair $x = \langle \Gamma, q \rangle$. Here, $\Gamma = [p_1, p_2, \ldots, p_m]$ constitutes a finite set of FOL formulas representing the given premises (i.e., background facts and deductive rules), and q is an FOL formula denoting the target conclusion [22]. The core objective of the LLM $f(\cdot)$ is to function as a deductive reasoner, determining whether the premises semantically entail the conclusion, denoted as $\Gamma \models q$. Formally, this entailment holds if and only if every interpretation (or world model) that satisfies all formulas in Γ also satisfies q [38]. In this paper, we adopt a three-valued entailment setting: *True* iff $\Gamma \models q$, *False* iff $\Gamma \models \neg q$, *Unknown* otherwise.

Logical Equivalence and Logical Semantic Preservation. To construct MRs, we leverage the foundational property of *logical equivalence*. Two formulas ϕ and ψ are logically equivalent, denoted as $\phi \equiv \psi$, if they have identical truth values under all possible interpretations [22]. In FOL, equivalence laws such as De Morgan's laws, the Law of Contraposition, and Double Negation Elimination constitute strict equivalence transformations. Suppose we apply these laws to transform a source test case $\langle \Gamma, q \rangle$ into a follow-up test case $\langle \Gamma', q \rangle$. Because FOL mathematically guarantees $\Gamma \equiv \Gamma'$, it strictly preserves the invariance of the entailment relationship:

$$\Gamma \models q \iff \Gamma' \models q$$

Consequently, FOL-grounded transformations preserve the underlying logical semantics. Since the transformed formulas are later translated into natural language, we further audit the generated follow-up cases empirically to account for possible realization errors. Under this design, reported metamorphic violations can be interpreted as logical reasoning defects unless they are attributable to translation drift or output parsing errors.

2.2. Metamorphic testing for large language models

In the paradigm of software testing, the System Under Test (SUT) is typically abstracted as a target function $f : X \rightarrow Y$, mapping an input space X to an output space Y. Traditional testing methodologies rely heavily on a *test oracle* to verify whether the actual execution output $f(x)$ matches the expected ground truth for a given input x [29]. However, for highly complex systems such as LLMs, acquiring an exact oracle for arbitrary inputs is prohibitively expensive, leading to the severe oracle problem [28,30]. MT alleviates this fundamental bottleneck by shifting the verification from individual test case to the properties among multiple executions.

The core of MT lies in defining MRs, which specify the necessary properties that the target function f must satisfy across multiple executions. Formally, given a tuple of n test inputs $(x_1, \ldots, x_n)$ where $n \geq 2$, an MR is defined as a logical implication:

$$\mathcal{R}_i(x_1, \ldots, x_n) \Rightarrow \mathcal{R}_o(f(x_1), \ldots, f(x_n))$$

where $\mathcal{R}_i$ represents the *input relation* among the inputs, and $\mathcal{R}_o$ represents the *output relation* expected to hold among the corresponding

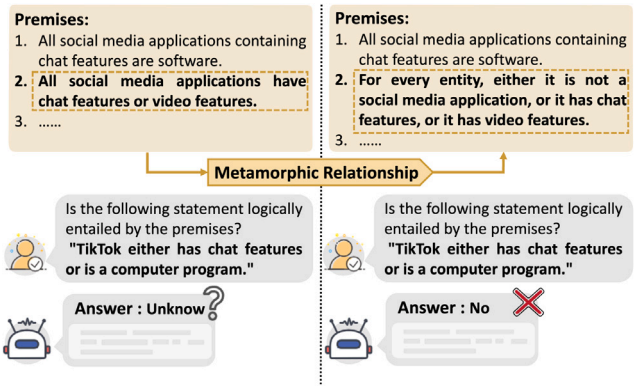


Fig. 1. An illustration of an LLM's failure to maintain consistency across logically equivalent reasoning tasks.

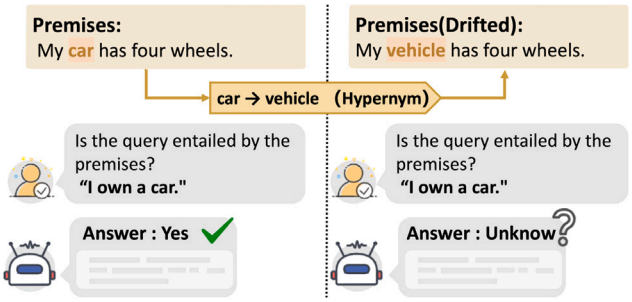


Fig. 2. An illustration of semantic drift caused by traditional lexical MRs.

outputs [34]. For the sake of conciseness in the following discussion, we denote a metamorphic relation as the pair $\langle \mathcal{R}_i, \mathcal{R}_o \rangle$.

In a standard MT workflow, testing begins with the selection of a *source test case* x_s. A specific input transformation is then applied to derive a *follow-up test case* x_f, strictly ensuring that the predefined input relation holds true (i.e., $\mathcal{R}_i(x_s, x_f) = \text{True}$). The combination of the source and follow-up test cases constitutes a *metamorphic test group (MG)*,[1] denoted as the tuple $\langle x_s, x_f \rangle$. This MG is subsequently fed into the LLM under test to yield the corresponding outputs $f(x_s)$ and $f(x_f)$. A *metamorphic oracle violation* occurs if and only if the inputs satisfy $\mathcal{R}_i$ while the corresponding outputs fail to satisfy $\mathcal{R}_o$. Consequently, a logical reasoning defect is detected when the following Boolean expression evaluates to True:

$$\mathcal{R}_i(x_s, x_f) \wedge \neg \mathcal{R}_o(f(x_s), f(x_f))$$

To illustrate this workflow, consider evaluating the logical reasoning capability of an LLM. Suppose we sample a source test case x_s formulated as a natural language query: "*Premise: All birds can fly. Tweety is a bird. Conclusion: Tweety can fly. Is this valid?*". Due to the absence of a test oracle, the correctness of the LLM's output $f(x_s)$ is unverifiable. However, by designing an MR based on the logical *Law of Contraposition*, we can transform x_s into a follow-up test case x_f: "*Premise: Anything that cannot fly is not a bird. Tweety is a bird. Conclusion: Tweety can fly. Is this valid?*". Given the strict logical equivalence guaranteed by the Law of Contraposition under FOL, the two queries pose the exact same reasoning task. Therefore, the expected output relation $\mathcal{R}_o$ requires that the LLM must yield semantically equivalent responses for both queries (e.g., affirming the validity of both). If the LLM predicts $f(x_s)$ to be valid but $f(x_f)$ to be invalid, a metamorphic oracle violation is triggered, capturing a genuine logical reasoning defect in the model. This example demonstrates how logic-based MT can expose internal reasoning inconsistencies without requiring ground truth. To provide a clear synthesis of how these elements align with the formal definitions of MT, we summarize the mapping between our formal notations and this concrete reasoning instance in Table 1.

2.3. Motivating example

In this section, we present concrete examples to illustrate the inherent limitations of current LLM evaluation paradigms. First, we highlight why MT is essential for exposing logically inconsistent reasoning that standard reference-based testing obscures. Second, we demonstrate why we design MRs based on FOL rather than lexical transformations to prevent semantic drift.

2.3.1. Reference-based testing vs. MT

Consider the logical entailment test case illustrated in Fig. 1. In the first query, the model is presented with a set of premises and a conclusion. The model answers *"Unknown"*, correctly aligning with the ground-truth label to indicate it cannot derive the conclusion from the given premises. In the second query, we modify only a single premise by applying an equivalence transformation. Specifically, the rule stating that *"all social media applications have chat features or video features"* is rewritten using implication elimination. All other premises, the conclusion, and the prompt format remain unchanged.

Under FOL, the transformed premise is logically equivalent to the original one. Therefore, the entailment relationship must remain unchanged. However, the model returns a logically inconsistent answer: *"No,"* indicating a clear failure to maintain consistency across equivalent logical reasoning tasks.

Traditional reference-based testing evaluates these queries in isolation against a ground-truth label, scoring them merely as "correct" or "incorrect." Such an evaluation obscures a critical issue: a single "correct" answer—such as the one achieved in the first query—may merely result from random guessing or superficial pattern matching rather than genuine logical reasoning. As demonstrated, one isolated success may provide a false sense of reasoning capability, failing to detect the underlying logical inconsistencies.

2.3.2. Logical vs. lexical MRs

Fig. 2 demonstrates how lexical MRs can induce semantic drift. First, we establish a baseline where the model correctly deduces the conclusion (*"I own a car."*) from a specific premise (*"My car has four wheels."*), returning an accurate judgment of *"Yes"*. Then, the premise is modified by replacing the target noun *"car"* with its broader hypernym, *"vehicle,"* while keeping the conclusion unchanged.

This minor substitution induces a semantic drift that invalidates the original deductive chain. Because the category of *"vehicle"* has a broader scope, a *"four-wheeled vehicle"* is not necessarily a car—it could realistically be a quadricycle or an ATV. Consequently, the modified premise no longer strictly entails the conclusion. Recognizing this logical difference, the LLM correctly updates its prediction to *"Unknown."*

To avoid this type of semantic drift, logical MRs change the transformation target. Instead of directly editing words in natural language, LGMT first transforms the underlying FOL representation using equivalence-preserving laws and then translates the transformed formula into natural language. Therefore, semantic preservation is guaranteed at the symbolic level, and the remaining risk is limited to the realization step from FOL to natural language, which we audit empirically.

[1] In classical MT literature, an MG encompasses the complete set of source and follow-up test cases required for one instance of checking the relevant MR. Since our MRs strictly involve a one-to-one mapping, we define the MG as a bi-tuple without loss of generality.

Table 1
Mapping of MT Component to the Logical Reasoning Case Study.

MT component	Notation	Instantiation in logical reasoning example
Source Test Case	x_s	"All birds can fly. Tweety is a bird. Conclusion: Tweety can fly."
Follow-up Test Case	x_f	"Anything that cannot fly is not a bird. Tweety is a bird. Conclusion: Tweety can fly."
Input Relation	$\mathcal{R}_i$	Logical equivalence via the *Law of Contraposition*.
Output Relation	$\mathcal{R}_o$	Semantic consistency between the LLM's responses $f(x_s)$ and $f(x_f)$.
Metamorphic Relation	$\langle \mathcal{R}_i, \mathcal{R}_o \rangle$	The constraint that queries transformed via the Law of Contraposition must yield semantically consistent reasoning results with the source queries.
Metamorphic Oracle Violation	$\mathcal{R}_i \wedge \neg\mathcal{R}_o$	The LLM confirms the validity for x_s but rejects it for x_f (or vice versa).

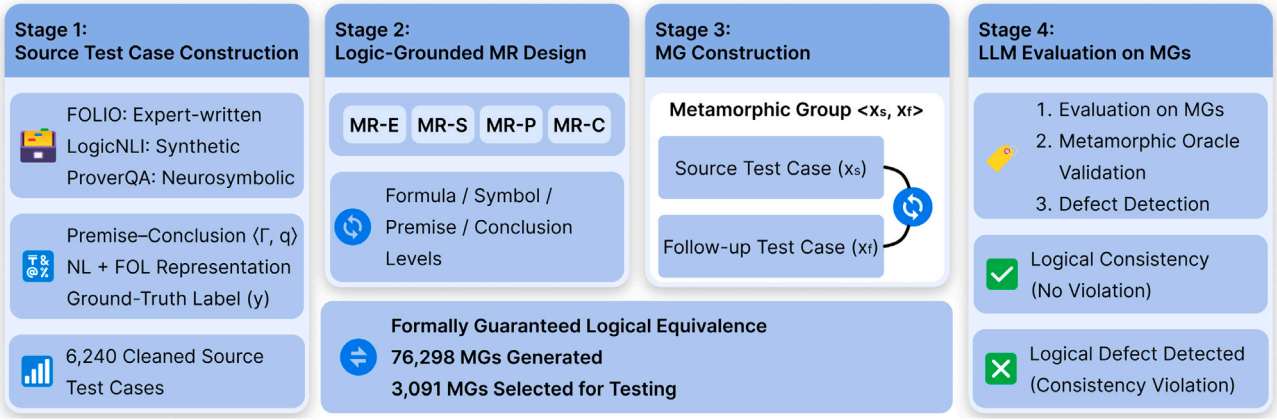


Fig. 3. Overview of the LGMT framework.

3. Methodology

In this section, we present **LGMT**, an automated framework designed to validate the logical reasoning reliability of LLMs. We first outline the testing workflow, then detail the core logic-grounded MRs, and finally describe the pipeline for constructing natural language test cases.

3.1. Framework overview

Fig. 3 illustrates the overall workflow of the proposed LGMT framework. LGMT is designed as a fully automated, oracle-free testing pipeline for evaluating the logical reasoning reliability of LLMs through logic-grounded metamorphic transformations and consistency checking. In this workflow, LGMT follows the oracle-free nature of MT: instead of requiring a ground-truth oracle for each generated follow-up test case, it checks whether model predictions satisfy the expected metamorphic relation. Accordingly, the detection of metamorphic oracle violations does not depend on benchmark labels, although such labels are retained for comparative experimental metrics such as static accuracy, consistent accuracy, HDR, and FUR.

The framework is organized into four sequential stages, aligned with the methodological components described in the following subsections.

(1) Source Test Case Construction. We construct source test cases from multiple FOL reasoning benchmarks (e.g., FOLIO, LogicNLI, and ProverQA). Each test case is represented in both natural language (premises and conclusion) and its corresponding FOL form, which serves as the basis for subsequent transformations.

(2) Logic-Grounded MR Design. We define a set of MRs grounded in first-order logic, including formula-level (MR-E), symbol-level (MR-S), premise-level (MR-P), and conclusion-level (MR-C) transformations. These MRs are formally guaranteed to preserve logical equivalence.

(3) Metamorphic Group Construction. Given a source test case, the selected MR is applied to its FOL representation to generate a transformed logical structure, which is then translated back into natural language. The source and transformed test cases together form a metamorphic group.

(4) LLM Evaluation and Defect Detection. Each MG is executed on the LLM under test. The model outputs are parsed into structured predictions and evaluated using a metamorphic oracle, which checks whether the expected consistency relation is preserved. Violations indicate logical reasoning defects.

Through this pipeline, LGMT shifts evaluation from isolated correctness to consistency under logic-preserving transformations, enabling scalable and oracle-free detection of reasoning failures. For clarity, we provide an end-to-end example of this pipeline in Appendix B.

3.2. Source test case construction

Each source test case for logical reasoning consists of a set of premises and a target conclusion. To support logic-grounded transformations, each source test case is represented in both natural language and its corresponding FOL form.

The construction of source test cases follows two key principles. First, the natural-language formulation must admit a faithful symbolic representation, ensuring that its logical structure can be accurately captured in FOL. Second, the symbolic representation must support the application of logic-preserving metamorphic relations, enabling systematic generation of follow-up test cases without altering the underlying semantics.

Accordingly, each source test case is normalized into a unified representation $\langle \Gamma, q, y \rangle$, where Γ denotes a finite set of premises, q is the target conclusion, and $y \in \{\text{True}, \text{False}, \text{Unknown}\}$ is the ground-truth label when available. While LGMT does not rely on ground-truth labels for defect detection, these labels are retained exclusively for quantitative evaluation and comparative analysis in the experimental study.

3.3. Logic-grounded MRs

To address the semantic drift induced by traditional lexical MT, we formulate 20 types of MRs strictly grounded in first-order logic. As summarized in Table 2, we systematically organize these MRs into

Table 2
Logic-grounded MRs.

Category	Subclass	# of MRs	Granularity	Transformation strategy
MR-E	MR-E1	6	Formula level	Constitutes a complete basis for PNNF (Appendix A).
	MR-E2	4	Formula level	Captures other valid logical equivalence transformations outside PNNF.
MR-S	–	2	Symbol level	Uniformly renames constants or predicates.
MR-P	–	5	Premise level	Structurally modifies Γ.
MR-C	–	3	Conclusion level	Structurally modifies q.

four distinct categories based on their *transformation granularity*: from atomic symbols up to the structure of the test case.

It is important to note that MR-E1 underlies the canonical normalization used in the PNNF completeness proof (as formally proved in Appendix A). The completeness claim here is *not* semantic completeness of first-order logic. Rather, it is completeness with respect to the PNNF-oriented normalization target used in this work: under the proposed rewrite procedure, any FOL formula can be rewritten into a canonical PNNF normal form using the atomic rewrite rules defined in MR-E1, and the obtained normal form is unique up to α-equivalence. In the context of LLM evaluation, this means that MR-E1 captures a core set of equivalence-preserving transformations required for systematic logical normalization.

We conduct the formal completeness analysis only for MR-E1 because MR-E1 is designed as the normalization-oriented core rewrite system. By contrast, MR-E2 contains additional valid equivalence laws outside this PNNF normalization basis, while MR-S, MR-P, and MR-C operate at the symbol, premise, and conclusion levels of a test case. These MRs are logic-preserving and useful for testing, but they are not intended to constitute a complete normalization procedure.

As established in Section 3.2, the logical skeleton of a test case is structured as a premise–conclusion pair (Γ, q). Throughout the subsequent formalizations, we use ϕ, ψ, and θ to represent arbitrary first-order formulas involved in our MRs. An MR transforms a source test case (Γ, q) into a follow-up test case (Γ', q'). We use the symbol $\rightsquigarrow$ to denote an application of an MR. Specifically, at the formula level, $\phi \rightsquigarrow \psi$ denotes a one-step rewrite of formulas induced by an MR; at the test-case level, $(\Gamma, q) \rightsquigarrow (\Gamma', q')$ denotes the corresponding transformation between test cases.

3.3.1. MR-e: Logical equivalence transformations

This group of MRs defines a set of logically equivalent rewrite rules for FOL formulas. Each MR is formulated as a rule schema that applies to arbitrary subformulas during logical normalization. As summarized in Table 2, this category comprises two subsets: MR-E1, which forms the mathematical basis of the PNNF completeness result (Theorem 1), and MR-E2, which captures additional valid equivalence rules not included in MR-E1.

MR-E1: Canonical Equivalence-Based MRs. This subset forms the mathematical basis of the PNNF completeness result (Theorem 1). It ensures that any formula can be reduced to a standard form:

MR-E1.1 (Implication and biconditional elimination). Implication and biconditional connectives are eliminated using the following rewrite rules:

$$\varphi \to \psi \rightsquigarrow \neg\varphi \vee \psi$$

$$\varphi \leftrightarrow \psi \rightsquigarrow (\neg\varphi \vee \psi) \wedge (\neg\psi \vee \varphi)$$

MR-E1.2 (Negation normalization). Negation is pushed inward until it applies only to atomic formulas through double negation, De Morgan's laws, and quantifier duality:

$$\neg\neg\varphi \rightsquigarrow \varphi$$

$$\neg(\varphi \wedge \psi) \rightsquigarrow (\neg\varphi) \vee (\neg\psi), \quad \neg(\varphi \vee \psi) \rightsquigarrow (\neg\varphi) \wedge (\neg\psi)$$

$$\neg\forall x\, \varphi \rightsquigarrow \exists x\, \neg\varphi, \quad \neg\exists x\, \varphi \rightsquigarrow \forall x\, \neg\varphi$$

MR-E1.3 (Quantifier lifting). To move quantifiers toward the prenex prefix, let $Q \in \{\forall, \exists\}$, $\circ \in \{\wedge, \vee\}$ and $\mathrm{FV}(\psi)$ to denote the free variables in ψ. Provided $x \notin \mathrm{FV}(\psi)$, the following lifting rules apply:

$$(Q x\, \varphi) \circ \psi \rightsquigarrow Q x\, (\varphi \circ \psi)$$

$$\psi \circ (Q x\, \varphi) \rightsquigarrow Q x\, (\psi \circ \varphi)$$

MR-E1.4 (Structural normalization). Conjunctions and disjunctions are flattened and reordered according to a fixed canonical ordering $\prec$ (e.g., lexicographic order):

$$(\varphi \wedge (\psi \wedge \theta)) \rightsquigarrow (\varphi \wedge \psi \wedge \theta)$$

$$(\varphi \vee (\psi \vee \theta)) \rightsquigarrow (\varphi \vee \psi \vee \theta)$$

$$(\varphi_1 \wedge \cdots \wedge \varphi_n) \rightsquigarrow (\varphi_{\sigma(1)} \wedge \cdots \wedge \varphi_{\sigma(n)})$$

where σ is the permutation that sorts the sequence by $\prec$.

MR-E1.5 (α-renaming). To resolve variable name conflicts that hinder quantifier lifting, we perform α-conversion. Let $\mathrm{Var}(\varphi)$ denote the set of all variables in φ:

$$(Q x\, \varphi) \circ \psi \rightsquigarrow (Q y\, \varphi[x := y]) \circ \psi$$

where $x \in \mathrm{Var}(\psi)$ and $y \notin \mathrm{Var}(\varphi) \cup \mathrm{Var}(\psi)$.

MR-E1.6 (Canonical quantifier ordering). Within a prenex prefix, adjacent identical quantifiers (blocks) are reordered according to the first occurrence of their variables in the matrix M:

$$Q x_1 \cdots Q x_r . M \rightsquigarrow Q x_{\sigma(1)} \cdots Q x_{\sigma(r)} . M$$

where σ ensures a canonical variable sequence within each quantifier block.

MR-E2: Non-Canonical Equivalence-Based MRs. While MR-E1 covers the complete set for canonical normalization, MR-E2 captures additional valid equivalence laws that further challenge the model's ability to simplify or expand logical expressions.

MR-E2.1 (Redundancy elimination). This type of MR leverages idempotence and absorption laws to eliminate duplicated or subsumed subformulas:

$$\varphi \vee \varphi \rightsquigarrow \varphi, \quad \varphi \wedge \varphi \rightsquigarrow \varphi$$

$$\varphi \vee (\varphi \wedge \psi) \rightsquigarrow \varphi, \quad \varphi \wedge (\varphi \vee \psi) \rightsquigarrow \varphi$$

MR-E2.2 (Tautology and contradiction elimination). This type of MR eliminates trivial formulas using the laws of excluded middle and contradiction:

$$\varphi \vee \neg\varphi \rightsquigarrow \mathrm{True}, \quad \varphi \wedge \neg\varphi \rightsquigarrow \mathrm{False}$$

MR-E2.3 (Identity and domination laws). This type of MR evaluates the model's capacity to prune logically inert or dominant components:

$$\varphi \wedge \mathrm{True} \rightsquigarrow \varphi, \quad \varphi \vee \mathrm{False} \rightsquigarrow \varphi$$

$$\varphi \vee \mathrm{True} \rightsquigarrow \mathrm{True}, \quad \varphi \wedge \mathrm{False} \rightsquigarrow \mathrm{False}$$

MR-E2.4 (Distributivity law). This MR rewrites formulas by redistributing conjunctions over disjunctions (and vice versa):

$$\varphi \wedge (\psi \vee \theta) \rightsquigarrow (\varphi \wedge \psi) \vee (\varphi \wedge \theta)$$

$$\varphi \vee (\psi \wedge \theta) \rightsquigarrow (\varphi \vee \psi) \wedge (\varphi \vee \theta)$$

3.3.2. MR-s: Symbol renaming invariance

This group of MRs captures invariance under systematic renaming of non-logical symbols, including constant symbols and predicate symbols. Such renaming operations preserve the logical form of formulas and the entailment relations between premises and conclusions, provided that the renaming is applied uniformly. Therefore, a logically sound reasoner is expected to produce identical entailment judgments before and after symbol renaming. We use the notation $(\Gamma, q)[s := s']$ to denote the test case obtained by uniformly replacing every occurrence of the symbol s in both Γ and q with the symbol s'.

MR-S1 (Constant Renaming Invariance). This type of MR captures invariance under uniform renaming of constant symbols, which correspond to object identifiers in the domain. A constant symbol is consistently replaced by a fresh constant symbol throughout the test case, while all other symbols remain unchanged. At the test-case level, the MR is defined as:

$$(\Gamma, q) \rightsquigarrow (\Gamma, q)[c := c']$$

where c is a constant symbol occurring in (Γ, q) and c' is a fresh constant symbol.

MR-S2 (Predicate Renaming Invariance). This type of MR captures invariance under uniform renaming of predicate symbols, which label relations over domain objects. A predicate symbol is consistently replaced by a fresh predicate symbol throughout the test case, while all other symbols remain unchanged. At the test-case level, the MR is defined as:

$$(\Gamma, q) \rightsquigarrow (\Gamma, q)[P := P']$$

where P is a predicate symbol occurring in (Γ, q) and P' is a fresh predicate symbol.

3.3.3. MR-P: Premise transformation

This group of MRs captures the invariance of entailment judgments under premise transformations which are inspired by Chen et al. [33]. At the test-case level, these MRs are defined as:

$$(\Gamma, q) \rightsquigarrow (\Gamma', q)$$

where Γ' is obtained from Γ by applying the corresponding premise transformation.

MR-P1 (Premise Reordering). The order of premises in Γ is permuted. This MR changes only the presentation order of premises and does not affect their logical content.

MR-P2 (Premise Duplication). A premise already occurring in Γ is duplicated and added again to the premise set. This introduces redundancy without adding new information.

MR-P3 (Irrelevant Extension). A logically irrelevant but non-contradictory premise is added to the premise set. Such a premise does not affect the entailment relation between Γ and q.

MR-P4 (Premise Fusion). Two premises $\varphi, \psi \in \Gamma$ are combined into a conjunctive premise $\varphi \wedge \psi$, which is added to the premise set. This transformation preserves information while altering the structural form of the premises.

MR-P5 (Premise Decomposition). A conjunctive premise $\varphi \wedge \psi \in \Gamma$ is decomposed into two separate premises φ and ψ, which are added to the premise set.

3.3.4. MR-C: Conclusion transformation

This group of MRs captures invariance of entailment judgments under structural transformations of the conclusion. At the test-case level, this group of MRs is defined as:

$$(\Gamma, q) \rightsquigarrow (\Gamma, q')$$

where q' is obtained from q by applying the corresponding conclusion transformation.

MR-C1 (Conjunction with Truth). The conclusion q is transformed into $q \wedge \text{True}$.

MR-C2 (Disjunction with Falsehood). The conclusion q is transformed into $q \vee \text{False}$.

MR-C3 (Double Negation). The conclusion q is transformed into $\neg\neg q$.

To make these MRs easier to apply in practice, Section 3.6 provides a compact example showing how the four categories of MRs are applied in LLM testing.

3.4. Metamorphic group construction

Building upon the MRs defined in Section 3.3, we apply these logical transformations to each source test case to generate its corresponding follow-up test case ($x_s \rightsquigarrow x_f$). As established in Section 3.2, our framework maintains a dual-representation system. Under this design, the application of an MR mutates the underlying FOL skeleton first (e.g., $\phi \rightsquigarrow \psi$, where $\phi \in \Gamma$).

To translate this mutated FOL formula back into natural language, we employ an LLM-based translation module. Crucially, the LLM is constrained via prompting to operate strictly as a parser and translator rather than a reasoner. This constrains the translation module to preserve the intended logical structure as much as possible, while avoiding additional reasoning or simplification. Finally, the source test case and its generated follow-up test case are coupled to form a complete metamorphic group for evaluation, formally defined as the tuple $\langle x_s, x_f \rangle$.

3.5. Metamorphic oracle and defect detection

The final module of the LGMT framework is automated test result verification. Given an MG $\langle x_s, x_f \rangle$, we systematically evaluate the reasoning robustness of the LLM under test, denoted as $f(\cdot)$. We feed x_s and x_f independently into the model to obtain their respective raw textual outputs, $f(x_s)$ and $f(x_f)$.

Because LLM outputs are generative, we parse these raw responses to extract discrete logical predictions (e.g., True, False, or Unknown), denoted as $\hat{y}_s$ and $\hat{y}_f$. The metamorphic oracle then acts as an automated, black-box verification mechanism by checking whether the relationship between $\hat{y}_s$ and $\hat{y}_f$ satisfies the expected output relation ($\mathcal{R}_o$) dictated by the applied MR.

Any deviation from the expected $\mathcal{R}_o$ constitutes a *metamorphic oracle violation*. Because our FOL-grounded MRs strictly preserve the underlying logic of source test cases, each violation indicates a potential reasoning inconsistency, after excluding translation and parsing artifacts.

3.6. Illustrative MR applications

Following the MR categorization by transformation granularity in Section 3.3 and Table 2, we provide an example to bridge the formal MR definitions and their practical use in LLM testing. To keep the discussion connected to the motivating example in Fig. 1, consider the following simplified source test case:

Premises: (1) All social media applications have chat features or video features. (2) TikTok is a social media application.

Conclusion: TikTok has chat features or video features.

Its FOL representation is $\Gamma = [\forall x(S(x) \rightarrow (C(x) \vee V(x))), S(t)]$ and $q = C(t) \vee V(t)$, where S denotes *social media application*, C denotes *has chat features*, V denotes *has video features*, and t denotes TikTok. The expected judgment is *True*. LGMT does not need this label to detect a metamorphic violation, and the label is shown here only to make the example intuitive.

Formula-level application. At the formula level, LGMT applies an equivalence rule to a formula in the test case. For example, MR-E1.1 rewrites the first premise by implication elimination:

$$\forall x(S(x) \rightarrow (C(x) \vee V(x))) \nrightarrow \forall x(\neg S(x) \vee C(x) \vee V(x)).$$

The corresponding follow-up case uses the premise *"For every application, either it is not a social media application, or it has chat features or video features"*, while keeping the second premise and the conclusion unchanged. Because the transformed premise is logically equivalent to the original one, a reliable reasoner should return the same label for the source and follow-up cases.

Symbol-level application. At the symbol level, LGMT uniformly renames non-logical symbols while preserving the inference structure. For example, MR-S can replace S, C, and V with fresh predicates P, Q, and R, and replace the constant t with a, throughout both Γ and q. A follow-up case can therefore be realized as: *"All objects with property P have property Q or property R. a has property P. Conclusion: a has property Q or property R."* The expected judgment must remain unchanged.

Premise-level application. At the premise level, LGMT modifies the organization of the premise set without changing its logical content. For example, MR-P1 can reorder the two premises, and MR-P2 can duplicate the premise *"TikTok is a social media application."* In both cases, the available information is unchanged, so the model should preserve its original judgment.

Conclusion-level application. At the conclusion level, LGMT applies an equivalence-preserving transformation to q. For example, MR-C2 rewrites the conclusion as $(C(t) \vee V(t)) \vee \text{False}$, which can be realized as *"TikTok has chat features or video features, or five times five equals one."* The follow-up conclusion is logically equivalent to the original conclusion, so the model's entailment judgment should remain the same.

This example illustrates the role of the four MR categories in LGMT. Formula-level MRs alter logical expressions, symbol-level MRs abstract away lexical cues, premise-level MRs modify the structure of the premise set, and conclusion-level MRs reformulate the query target. In all cases, the expected output relation is consistency between the source and follow-up predictions.

4. Experimental setup

In this section, we present the experimental design for evaluating the effectiveness of our LGMT framework. We first outline the core research questions guiding our empirical study, then detail the selection of models under test, the test datasets and sampling strategy, and finally describe the evaluation metrics and analysis methods.

4.1. Research questions

To systematically and rigorously evaluate our framework, we organize our empirical study around four core research questions, inspired by recent state-of-the-art evaluation paradigms for MT for LLMs [34]:

- **RQ1 (Effectiveness):** How effectively can LGMT detect logical reasoning defects in LLMs?
- **RQ2 (Comparison):** How does LGMT compare with traditional reference-based testing?
- **RQ3 (Sensitivity):** How do different categories of logic-grounded MRs impact the inconsistency detection rate?
- **RQ4 (Mitigability):** Can advanced prompting strategies, such as Few-shot and Few-shot CoT prompting, mitigate the logical defects exposed by LGMT?

Table 3
Summary of the evaluated models.

Category	Model name	API identifier
Proprietary	GPT-5.2	`gpt-5.2`
	Claude 4.5 Sonnet	`claude-4-5-sonnet`
Open-Weights	Llama 3.3 (70B)	`Llama-3.3-70B-Instruct`
	DeepSeek Chat	`deepseek-chat`
Reasoning	OpenAI o4-mini	`o4-mini`
	DeepSeek Reasoner	`deepseek-reasoner`

These RQs are designed to comprehensively validate LGMT from both testing and evaluation perspectives. **RQ1** assesses the overall fault-detection effectiveness of LGMT (i.e., metamorphic oracle violation rate). **RQ2** compares our metamorphic oracle against traditional reference-based testing methods to expose hidden defects that successfully evade traditional reference-based evaluations. **RQ3** provides a sensitivity analysis across different categories of our logic-grounded MRs. Finally, **RQ4** investigates whether advanced prompting strategies (e.g., *Zero-shot*, *Zero-shot CoT*, *Few-shot*, and *Few-shot CoT*) can effectively mitigate the exposed logical defects.

4.2. Models under test

To systematically evaluate the generalizability of our LGMT framework, we carefully selected six state-of-the-art LLMs. As shown in Table 3, these models are categorized into three distinct categories: *Proprietary Models*, *Open-Weights Models*, and *Reasoning Models* (which are specifically optimized for complex reasoning tasks). Proprietary and reasoning models were accessed via their official API providers, whereas standard open-weights models were hosted on high-performance inference platforms (e.g., Fireworks AI).

To ensure a fair and deterministic evaluation across these diverse hosting environments, we strictly set `temperature=0` (greedy decoding) for all models. This eliminates the inherent stochasticity of LLM generation, ensuring that the detected metamorphic oracle violations reflect structural reasoning defects rather than random sampling variations. The only exception is for reasoning models, where the temperature parameter is inherently restricted by the official API design.

4.3. Test datasets and sampling strategy

To instantiate the source test case construction process described in Section 3.2, we derive source test cases from three diverse FOL reasoning benchmarks: FOLIO [21], LogicNLI [39], and ProverQA [25]. To ensure the quality of our test cases, we applied rigorous data cleaning and filtration criteria tailored to each dataset. Specifically, for FOLIO, we extracted 995 instances and filtered out those with unparseable logical expressions. For LogicNLI, we developed a customized logical solver to identify and eliminate instances with inherently contradictory premises, ultimately yielding 4245 high-quality source test cases. For ProverQA, we utilized the complete set of 1000 instances. By applying our 20 types of logic-grounded MRs to these source test cases, our LGMT framework automatically generated a large candidate pool of 76,298 MGs.

However, executing all 76,298 MGs across multiple LLMs is computationally prohibitive. To balance computational feasibility with strict statistical guarantees, we employed a sampling strategy. According to Cochran's formula [40], a sample size of $N \geq 385$ ensures a margin of error within 5% at a 95% confidence level. To satisfy this threshold for every major analytical category (MR-E, MR-S, MR-P, and MR-C) while ensuring fine-grained diversity, we randomly sampled up to 200 instances per sub-rule (e.g., MR-S1, MR-S2). Because each major category comprises at least two sub-rules, this strategy naturally aggregates to a minimum of 400 instances per category (e.g., MR-S yields $2 \times 200 = 400$).

Table 4
Distribution of sampled MGs across MR categories.

MR category	Sampled MGs	Percentage
MR-E	1091	35.30%
MR-S	400	12.94%
MR-P	1000	32.35%
MR-C	600	19.41%
Total	3091	100.00%

Finally, our evaluation test dataset constitutes a total of 3091 MGs. The distribution of sampled MGs across the four MR categories is shown in Table 4.

It is important to note that due to strict structural applicability constraints, certain MRs yielded fewer than the 200-instance maximum. Nevertheless, we manually confirmed that the aggregated sample size for every major category safely remains well above the 385 threshold (specifically, the smallest category comprises exactly 400 cases).

4.4. Prompting strategies

When evaluating LLMs, a common concern is that test failures might simply result from inadequate prompting rather than genuine reasoning defects. To address this (RQ4), we evaluate the models under four established paradigms: Zero-shot, Zero-shot CoT, Few-shot, and Few-shot CoT. While *Zero-shot CoT* serves as our default setting (RQ1–RQ3), contrasting it with the other paradigms allows us to investigate whether Few-shot and CoT prompting strategies can effectively mitigate these logical defects.

4.5. Evaluation metrics

To systematically address our Research Questions, we define three categories of evaluation metrics, precisely aligned with our testing objectives and comparative paradigms.

4.5.1. Metrics for fault detection effectiveness (rq1, rq3, RQ4)

To evaluate the absolute capability of LGMT in exposing logical reasoning defects without relying on ground-truth labels, we employ *Metamorphic Oracle Violation Rate (MVR)*, which is defined as the proportion of metamorphic groups where the LLM's outputs violate the corresponding metamorphic relation [34]. MVR directly quantifies the inconsistency detection rate of LGMT in exposing logical reasoning defects.

4.5.2. Metrics for LGMT reliability and validity (RQ1)

As established in Section 3, the theoretical soundness of our testing framework is guaranteed by our FOL-grounded approach. To empirically validate this, we define *False Report Rate (FRR)* as the proportion of reported metamorphic oracle violations that are actually false positives. To quantify this, we categorize false positives into two types based on their root causes.

The first type corresponds to violations caused by *semantic drift*, where the natural-language realization of the follow-up test case deviates from its underlying FOL representation, resulting in a mismatch between the intended logical transformation and the actual input semantics.

The second type corresponds to violations caused by *parsing errors*, where the LLM's raw output is incorrectly mapped to a discrete logical label. In these cases, the inconsistency arises from output misinterpretation rather than the model's reasoning behavior. Specifically, all evaluation prompts require a strict JSON output with a `label` field. If the output cannot be parsed into one of the three valid labels, or if the parsed label is ambiguous, the case is logged with its raw output and MG identifier.

Evaluated through rigorous manual inspection on a random sample of N reported violations, FRR is calculated as:

$$FRR = \frac{FP_{drift} + FP_{parse}}{N}$$

where FP_{drift} and FP_{parse} denote the numbers of false positives caused by semantic drift and parsing errors, respectively, and N is the total number of inspected reported violations.

4.5.3. Metrics for comparison and analysis (RQ2, RQ4)

To benchmark LGMT against traditional reference-based testing, we utilize ground-truth labels to define the following metrics:

- **Static Accuracy (Acc_{static}):** The standard accuracy of the models on the source test cases, evaluated strictly against ground-truth labels. This metric captures the model's apparent reasoning capability under the traditional static reference-based testing.
- **Consistent Accuracy (Acc_{cons}):** The proportion of metamorphic test groups where the models correctly answer both the source test case and the follow-up test cases. This metric reduces the impact of coincidental correctness by requiring consistent predictions across logically equivalent test cases.
- **Hidden Defect Rate (HDR):** HDR is defined as the proportion of test cases that yield correct predictions under traditional static evaluation (Acc_{static}) but trigger a metamorphic oracle violation. This metric quantifies the performance degradation from Acc_{static} to Acc_{cons}, reflecting the extent to which static benchmarks overestimate model capabilities due to hidden defects that evade traditional evaluation but are exposed by LGMT.

4.5.4. Metrics for undetected logical errors (RQ4)

While LGMT detects logical inconsistencies through metamorphic oracle violations, it cannot capture cases where the model produces identical but incorrect predictions across both the source and follow-up test cases. To quantify this limitation, we define the *False Unreported Rate (FUR)* as the proportion of metamorphic groups where both answers are incorrect but no metamorphic oracle violation is triggered. FUR characterizes a fundamental blind spot of consistency-based testing, where models may exhibit consistent but incorrect predictions. We note that FUR is not equivalent to the classical false negative rate, as it is defined with respect to benchmark ground-truth labels and only captures a subset of undetected errors.

5. Experimental results and analysis

In this section, we present the empirical results and analysis of our LGMT framework to answer the four research questions formulated in Section 4.1.

5.1. Answer to RQ1

RQ1 investigates whether LGMT can effectively expose logical reasoning defects in LLMs while maintaining a low false report rate. To answer this question, we evaluate LGMT from two complementary perspectives: *effectiveness*, measured by the Metamorphic Oracle Violation Rate (MVR), and *reliability*, measured by the False Report Rate (FRR).

As shown in Table 5, LGMT detects a substantial number of metamorphic oracle violations across all evaluated models, with MVR ranging from 20.84% to 29.44%. DeepSeek Reasoner exhibits the highest violation rate (29.44%), followed by GPT-5.2 (26.76%), while Claude 4.5 Sonnet shows the lowest MVR (20.84%).

Finding 1.1: All evaluated models exhibit substantial logical reasoning defects, and the observed violation rates do not consistently correlate with model performance or reasoning-oriented design.
Implication: Although Claude 4.5 Sonnet achieves the lowest MVR (20.84%), its violation rate remains non-trivial. GPT-5.2 still exhibits

Table 5
Overall effectiveness and reliability of LGMT.

Model	MVR	# Violations
GPT-5.2	26.76%	827
Claude 4.5 Sonnet	20.84%	644
Llama 3.3	25.43%	786
DeepSeek Chat	24.72%	764
o4-mini	22.06%	682
DeepSeek Reasoner	29.44%	910

Table 6
Reliability of LGMT based on audited violations.

Sample Size	FP_{drift}	FP_{parse}	FRR
360	4	1	1.38%

a relatively high MVR of 26.76%, while the two reasoning-oriented models show markedly different robustness: o4-mini performs comparatively well (22.06%), but DeepSeek Reasoner records the highest MVR overall (29.44%). These results suggest that the observed violation rates do not consistently track overall model performance or reasoning-oriented design.

To assess whether these reported violations correspond to genuine reasoning defects, we further audited a subset of cases using an LLM-assisted workflow with human verification. Instead of sampling per model, we uniformly sampled 360 reported violations from all 4613 MG violations, which satisfies Cochran's sampling criterion [40]. In the first stage, a strong LLM (GPT-5.4) was used as a preliminary auditor to screen whether each generated follow-up test case faithfully preserved the intended logical meaning of its transformed FOL formula. Among the 360 sampled cases, 54 were flagged as potentially problematic. In the second stage, all flagged cases were manually verified.

Finding 1.2: Only a small fraction of reported violations are attributable to semantic drift or parsing errors, indicating that LGMT reports are highly reliable.
Implication: Manual inspection confirms that only 4 cases exhibit genuine semantic drift (FP_{drift}), and these are mainly caused by inaccuracies in the FOL-to-natural-language translation process rather than flaws in the underlying logic-grounded transformations. Across all audited cases, only one parser-induced false positive was observed. As summarized in Table 6, this corresponds to an empirical FRR of 1.38%, indicating that the vast majority of reported violations reflect genuine logical reasoning defects rather than artifacts of test generation.

Overall, these results confirm that LGMT is both effective and reliable: it reveals substantial logical reasoning defects across all evaluated models, while the low empirical FRR indicates that these reported violations overwhelmingly correspond to genuine defects rather than artifacts of test generation.

5.2. Answer to RQ2

For RQ2, we investigate whether traditional reference-based testing overestimates the logical reasoning capabilities of LLMs by comparing it with LGMT from both apparent correctness and consistency-preserved correctness. Specifically, Acc_{static} measures model performance under traditional reference-based evaluation, whereas Acc_{cons} measures whether such correctness can be preserved under logic-preserving metamorphic transformations. The overall comparison, together with the Hidden Defect Rate (HDR), is summarized in Table 7.

Finding 2.1: Under traditional reference-based testing, all evaluated models achieve relatively high static accuracy (65.38%–73.53%), suggesting strong reasoning capability.
Implication: Static benchmark evaluation presents an overall optimistic view of LLM logical reasoning ability and suggests a model ranking dominated by proprietary models.

Table 7
Comparison of traditional reference-based testing and LGMT.

Model	Acc_static	Acc_cons	HDR
GPT-5.2	73.34%	56.84%	16.50%
Claude 4.5 Sonnet	73.53%	60.97%	12.56%
Llama 3.3	65.38%	52.38%	13.01%
DeepSeek Chat	69.91%	55.65%	14.27%
o4-mini	69.36%	56.20%	13.17%
DeepSeek Reasoner	72.44%	56.71%	15.72%

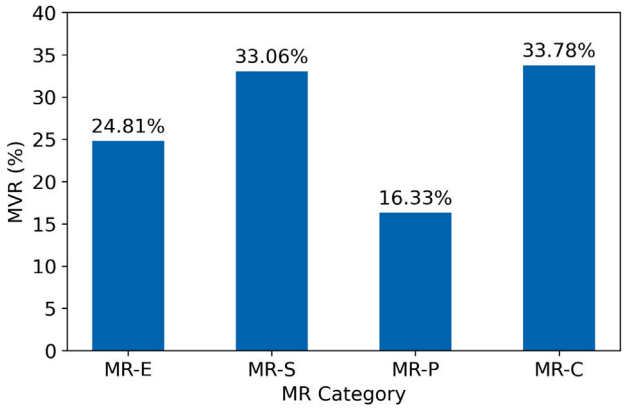


Fig. 4. Average MVR across MR categories. MR-C and MR-S induce the highest inconsistency, while MR-P shows substantially lower sensitivity.

Finding 2.2: Under LGMT, all models exhibit a clear drop from Acc_{static} to Acc_{cons}, with consistent accuracy decreasing to 52.38%–60.97%; for example, GPT-5.2 drops from 73.34% to 56.84%, Claude 4.5 Sonnet from 73.53% to 60.97%, and DeepSeek Reasoner from 72.44% to 56.71%.
Implication: Reference-based testing captures only isolated correctness on source test cases, whereas LGMT reveals that such correctness is often not logically stable under logic-preserving metamorphic transformations. This indicates that many seemingly correct predictions are achieved through coincidental correctness rather than robust logical reasoning.

Finding 2.3: LGMT further differentiates models in terms of hidden defects and consistency, with Claude 4.5 Sonnet achieving the highest Acc_{cons} (60.97%) and the lowest HDR (12.56%), Llama 3.3 showing the weakest consistent performance (52.38%), and GPT-5.2 exhibiting the highest HDR (16.50%) despite its strong static accuracy.
Implication: LGMT not only lowers performance estimates, but also reshapes comparative judgments of model reliability beyond what static accuracy alone can reveal.

Taken together, these results demonstrate that traditional reference-based testing systematically overestimates the reasoning reliability of LLMs by counting isolated correct answers as evidence of reasoning ability. In contrast, LGMT evaluates whether such correctness can survive logic-preserving transformations, revealing that a substantial portion of these correct answers are in fact logically unstable.

5.3. Answer to RQ3

RQ3 investigates how different categories of logic-grounded MRs affect the detection rate of LGMT. Specifically, we analyze the Metamorphic Oracle Violation Rate (MVR) across four MR categories (MR-E, MR-S, MR-P, and MR-C) to understand which types of MRs are most challenging for LLMs.

Fig. 4 presents the average MVR across MR categories, while Table 8 provides a detailed breakdown across models. For completeness, we further report fine-grained MVR results at the individual MR level in Appendix C, which corroborate the category-level trends observed here.

Table 8
Sensitivity of LGMT across MR categories. Each cell reports the MVR. The Avg. MVR column is computed by aggregating violations across all models.

Category	Avg. MVR	GPT-5.2	Claude 4.5 Sonnet	Llama 3.3	DeepSeek Chat	o4-mini	DeepSeek reasoner
MR-E	24.81%	25.02%	21.36%	25.02%	24.93%	24.66%	27.86%
MR-S	33.06%	32.00%	31.83%	35.25%	35.00%	27.75%	36.50%
MR-P	16.33%	19.10%	10.60%	16.10%	15.90%	14.40%	21.90%
MR-C	33.78%	39.17%	29.67%	35.17%	32.17%	26.33%	40.17%

As shown in Fig. 4, the average MVR varies substantially across MR categories. MR-C and MR-S induce the highest inconsistency (33.78% and 33.06%, respectively), while MR-P results in the lowest MVR (16.33%). MR-E exhibits a moderate level of inconsistency (24.81%), indicating a clear separation between different types of logical transformations.

This pattern is consistently observed across all evaluated models, as shown in Table 8. For every model, MR-C and MR-S remain the most challenging categories, while MR-P consistently yields the lowest violation rates. This indicates that the observed sensitivity is not model-specific, but reflects a shared weakness across current LLMs.

The observed sensitivity differences can be attributed to the nature of the underlying transformations. The high MVR under MR-C suggests that LLMs are particularly vulnerable to equivalence-preserving transformations on conclusions, indicating instability in final decision-making. Similarly, the high MVR under MR-S reveals limited robustness to symbol renaming, suggesting that models may rely on lexical-level representations rather than fully abstract logical structures. In contrast, the comparatively lower MVR under MR-P indicates that premise-level structural transformations are less disruptive. However, the violation rate still exceeds 10% across all models, highlighting that LLM reasoning remains fundamentally unstable.

A representative MR-S2 failure case makes this result intuitive. In this case, the source test concerns a text summarization model trained with machine learning algorithms. The LLM correctly predicts *True* for the source case, but changes its prediction to *Unknown* after MR-S2 uniformly replaces the meaningful predicates with abstract symbols, even though the logical structure and expected label remain unchanged. This failure suggests that the model does not fully understand and reason over the underlying logical structure; instead, it relies on lexical cues in meaningful predicate names when producing its output. The full source and follow-up cases are provided in Appendix C.1.

Finding 3.1: LLMs exhibit significantly different sensitivity across MR categories, with MR-C and MR-S inducing the highest inconsistency, while MR-P results in the lowest violation rate.
Implication: This result suggests that many current LLMs have not learned reasoning procedures that are invariant to logically equivalent reformulations. Instead, their apparent correctness may still depend strongly on how a conclusion is phrased or how symbols are instantiated.

Finding 3.2: The sensitivity pattern across MR categories is highly consistent among all evaluated models.
Implication: This suggests that the observed weaknesses are not model-specific, but reflect a fundamental limitation in current LLMs when handling logic-preserving transformations.

Overall, these results demonstrate that LGMT not only detects logical inconsistencies, but also serves as a diagnostic tool for identifying the specific reasoning dimensions in which LLMs are most vulnerable.

5.4. Answer to RQ4

RQ4 investigates whether advanced prompting strategies can mitigate the logical defects exposed by LGMT. To this end, we evaluate models under four prompting settings (Zero-shot, Zero-shot CoT, Few-shot, and Few-shot CoT) and analyze their performance using both reference-based metrics (Acc_{static}, Acc_{cons}, HDR) and LGMT-based metrics (MVR and FUR), as summarized in Table 9. Figs. 5 and 6 further visualize the distribution of HDR and FUR across models and prompting strategies.

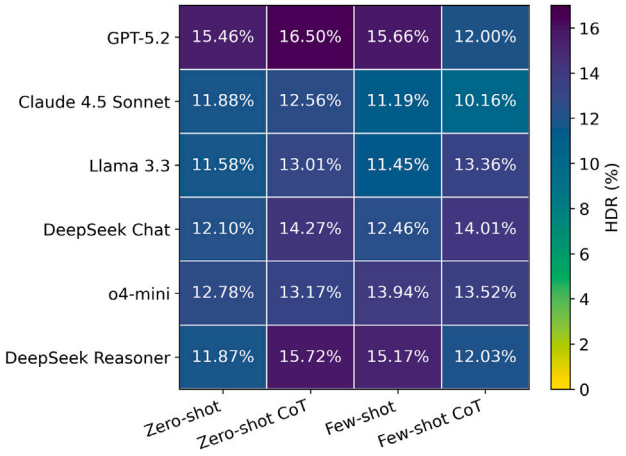


Fig. 5. Hidden Defect Rate (HDR) across models and prompting strategies.

Finding 4.1: Advanced prompting strategies generally improve both static accuracy and consistent accuracy, but the improvement is not monotonic across all models.
Implication: Across most models, Few-shot CoT achieves the best overall performance. For example, GPT-5.2 improves from Acc_{cons} 53.61% (Zero-shot) to 63.25% (Few-shot CoT), and DeepSeek Reasoner improves from 49.89% to 69.78%. However, CoT alone does not consistently yield improvements. For instance, Llama 3.3 shows a decrease from 55.68% to 52.38% when moving from Zero-shot to Zero-shot CoT, and DeepSeek Chat exhibits a slight drop under Few-shot CoT. This indicates that the effectiveness of prompting strategies depends on the model's capability to properly internalize and utilize reasoning instructions.

Finding 4.2: CoT prompting alone often increases hidden defects, while Few-shot CoT more effectively mitigates them.
Implication: As shown in Fig. 5, HDR tends to increase under Zero-shot CoT across most models (e.g., GPT-5.2: 15.46% → 16.50%, DeepSeek Reasoner: 11.87% → 15.72%), suggesting that unconstrained reasoning chains may introduce additional instability. In contrast, Few-shot CoT yields the lowest HDR for several models (e.g., GPT-5.2: 12.00%, Claude: 10.16%), indicating that demonstration-based prompting provides a more reliable bias for structured reasoning. Nevertheless, the reduction remains limited, implying that prompting can only partially alleviate hidden defects.

Finding 4.3: The effectiveness of prompting in reducing metamorphic oracle violations depends on model type, with better-aligned proprietary and reasoning-optimized models benefit more from Few-shot CoT.
Implication: For proprietary and reasoning models (e.g., GPT-5.2, Claude, DeepSeek Reasoner), MVR increases under Zero-shot CoT but decreases under Few-shot CoT, suggesting that examples help stabilize reasoning behavior. In contrast, open-weight models such as Llama 3.3 and DeepSeek Chat exhibit increased MVR even under Few-shot CoT, indicating that additional reasoning instructions may amplify

Table 9
Impact of prompting strategies on model performance. Acc_static, Acc_cons, and HDR are comparative evaluation metrics; MVR measures the defect detection capability of LGMT; FUR captures the limitation of consistency-based testing.

Model	Prompt	Acc_static	Acc_cons	HDR	MVR	FUR
GPT-5.2	Zero-shot	69.07%	53.61%	15.46%	23.07%	23.33%
GPT-5.2	Zero-shot CoT	73.34%	56.84%	16.50%	26.76%	16.40%
GPT-5.2	Few-shot	66.13%	50.47%	15.66%	23.65%	25.88%
GPT-5.2	Few-shot CoT	75.25%	63.25%	12.00%	18.57%	18.18%
Claude 4.5 Sonnet	Zero-shot	73.46%	61.59%	11.88%	20.52%	17.90%
Claude 4.5 Sonnet	Zero-shot CoT	73.53%	60.97%	12.56%	20.84%	18.19%
Claude 4.5 Sonnet	Few-shot	76.87%	65.67%	11.19%	19.15%	15.17%
Claude 4.5 Sonnet	Few-shot CoT	80.03%	69.87%	10.16%	17.22%	12.91%
Llama 3.3	Zero-shot	67.26%	55.68%	11.58%	20.48%	23.84%
Llama 3.3	Zero-shot CoT	65.38%	52.38%	13.01%	25.43%	22.19%
Llama 3.3	Few-shot	69.56%	58.10%	11.45%	19.83%	22.06%
Llama 3.3	Few-shot CoT	71.95%	58.59%	13.36%	23.10%	18.31%
DeepSeek Chat	Zero-shot	68.36%	56.26%	12.10%	22.29%	21.45%
DeepSeek Chat	Zero-shot CoT	69.91%	55.65%	14.27%	24.72%	19.64%
DeepSeek Chat	Few-shot	71.79%	59.33%	12.46%	19.99%	20.67%
DeepSeek Chat	Few-shot CoT	72.89%	58.88%	14.01%	25.98%	15.14%
o4-mini	Zero-shot	68.07%	55.29%	12.78%	22.06%	22.65%
o4-mini	Zero-shot CoT	69.36%	56.20%	13.17%	22.06%	21.74%
o4-mini	Few-shot	77.71%	63.77%	13.94%	22.39%	13.85%
o4-mini	Few-shot CoT	81.07%	67.55%	13.52%	21.81%	10.64%
DeepSeek Reasoner	Zero-shot	61.76%	49.89%	11.87%	20.67%	29.44%
DeepSeek Reasoner	Zero-shot CoT	72.44%	56.71%	15.72%	29.44%	13.85%
DeepSeek Reasoner	Few-shot	77.09%	61.92%	15.17%	26.08%	12.00%
DeepSeek Reasoner	Few-shot CoT	81.82%	69.78%	12.03%	20.45%	9.77%

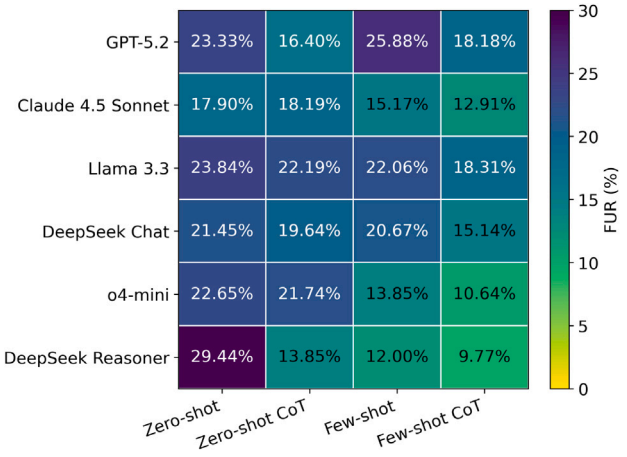


Fig. 6. False Unreported Rate (FUR) across models and prompting strategies.

instability when the model lacks sufficient reasoning alignment. This divergence suggests that the effectiveness of advanced prompting is conditioned on the model's inherent reasoning capability.

Finding 4.4: Few-shot CoT generally achieves the lowest FUR, but non-trivial blind spots remain.
Implication: As shown in Fig. 6, FUR is substantially reduced under Few-shot CoT for all models (e.g., DeepSeek Reasoner: 29.44% → 9.77%, o4-mini: 22.65% → 10.64%), indicating that fewer errors remain hidden as consistent-but-incorrect predictions. Combined with the low false report rate observed in RQ1, this suggests that LGMT is capable of exposing a large proportion of meaningful logical defects with high precision. However, FUR remains non-negligible (e.g., GPT-5.2: 18.18%, Llama 3.3: 18.31%), demonstrating that a significant fraction of errors still evade detection due to consistent but incorrect reasoning.

Overall, these results indicate that while advanced prompting strategies—especially Few-shot CoT—can partially mitigate the logical defects exposed by LGMT, they do not fundamentally eliminate reasoning inconsistencies. In particular, the persistence of non-trivial HDR and FUR values suggests that improvements in prompting primarily enhance apparent and consistent performance, rather than addressing the root causes of logical reasoning failures.

6. Discussion

6.1. Why Do LLMs struggle with logical invariance?

Our empirical results reveal that current LLMs exhibit significant sensitivity to logic-preserving transformations, particularly under MR-S and MR-C. These transformations do not alter the underlying semantics, yet they consistently induce high violation rates across all models. This suggests that LLMs do not rely on strictly logical reasoning, but instead rely heavily on lexical-level patterns and lexical cues.

Such reliance on surface representations can explain several observations in our results. In particular, the high sensitivity to symbol renaming suggests that models associate meaning with specific tokens, rather than encoding the underlying relational structure. Second, the instability under conclusion transformations suggests that final decision-making is not derived from a rigorous logical reasoning process, but is instead influenced by how the conclusion is phrased. Finally, the fact that Zero-shot CoT often increases inconsistency further indicates that generating longer reasoning chains does not guarantee more stable reasoning; instead, it may introduce additional opportunities for error propagation.

Taken together, these findings suggest that current LLMs approximate logical reasoning through pattern matching rather than performing robust symbolic or structure-aware inference. As a result, even logically equivalent formulations can lead to divergent predictions, exposing a fundamental limitation in their reasoning reliability.

6.2. Implications of logic-grounded MT

Our results highlight a key limitation of traditional reference-based testing: correctness on isolated test cases does not imply robustness under logically equivalent transformations. By shifting the evaluation from *correctness* to *invariance*, LGMT provides a fundamentally different perspective on model reliability.

In particular, LGMT evaluates whether a model can preserve its predictions under formally guaranteed logic-preserving transformations. This requirement goes beyond static correctness, because the model must preserve its judgment across multiple logically equivalent test cases of the same reasoning task, rather than merely answering one isolated query correctly. As demonstrated in RQ2, many predictions that appear correct under static evaluation fail to generalize under such transformations, indicating that they arise from coincidental correctness rather than genuine reasoning.

Moreover, grounding MRs in FOL ensures that all follow-up test cases are semantically equivalent to their source test cases, thereby substantially reducing the risk of semantic drift compared with lexical MRs. This guarantees that detected violations correspond to genuine reasoning defects, rather than artifacts of imperfect test generation.

Therefore, LGMT can be viewed as a principled and scalable alternative to traditional benchmarks, enabling systematic and oracle-free evaluation of logical reasoning robustness in LLMs.

6.3. Limitations and future directions

Despite its effectiveness, LGMT has inherent limitations. Most notably, it cannot detect cases where a model produces consistent but incorrect predictions across both source and follow-up test cases. This limitation is quantified by the FUR in RQ4, which remains non-negligible even under the strongest prompting settings. It indicates that consistency alone is insufficient to guarantee correctness, and that a portion of logical errors can still evade detection.

Future work can strengthen the testing protocol in two directions. First, *multi-MR validation* can extend the current pairwise source-follow-up design by pairing a single source test case with multiple logically equivalent follow-up variants derived from different MRs. This would enable stronger groupwise consistency checking and make it more difficult for locally stable but incorrect predictions to evade detection. Second, *MR composition* can apply multiple logic-grounded transformations sequentially or jointly to the same source test case. This would generate more structurally diverse yet logically equivalent follow-up cases and impose a stronger test of whether the model can preserve logical consistency under more complex reformulations.

Overall, while LGMT significantly improves the detection of logical reasoning defects, achieving truly robust logical reasoning in LLMs remains an open challenge.

6.4. Threats to validity

Construct validity. LGMT detects prediction inconsistency rather than absolute correctness. Therefore, a model that gives the same but incorrect answer to both the source and follow-up test cases may evade detection. We quantify this limitation using FUR. In addition, MVR depends on correct output parsing. To reduce this threat, we log parser failures and include parser-induced false positives in FRR.

Internal validity. Although our symbolic transformations preserve logical semantics at the FOL level, the FOL-to-natural-language realization stage may introduce semantic drift. To mitigate this threat, we constrain the translation prompt to preserve logical structure and conduct an LLM-assisted audit with human verification. The observed FRR is low, but translation errors cannot be fully eliminated.

External validity. Our evaluation focuses on FOL-style logical entailment tasks derived from FOLIO, LogicNLI, and ProverQA. The findings may not directly generalize to other reasoning settings, such as commonsense reasoning, mathematical problem solving, or long-horizon planning. Nevertheless, logical entailment provides a controlled setting for studying reasoning consistency under formally defined transformations.

Conclusion validity. The results may be affected by model versions, API implementations, and prompting choices. We reduce randomness by using deterministic decoding whenever supported and evaluate four prompting strategies to assess prompt sensitivity. However, future model updates may change the absolute violation rates.

7. Related work

In this section, we review related work from three perspectives most relevant to our study: logical reasoning evaluation for LLMs, metamorphic testing for NLP and LLMs, and neurosymbolic approaches that combine language models with formal logic.

7.1. Evaluating logical reasoning in LLMs

Logical reasoning is a fundamental capability that directly affects the reliability of intelligent systems. Traditionally, the NLP community has relied on static benchmarks to evaluate this ability, including datasets such as ARC [41], LogiQA [23], ReClor [19], and FOLIO [21]. With the rise of LLMs, recent studies have conducted broader evaluations of reasoning ability from multiple perspectives. Xu et al. [15] examine LLMs from deductive, inductive, and abductive perspectives, showing that fluent generation does not necessarily imply reliable logical inference. Similarly, LogicBench [22] and Multi-LogiEval [42] evaluate LLMs against formal inference rules and multi-step reasoning depth. Other efforts, such as LogiEval [18] and LogicAsker [16], further broaden the evaluation scope and reveal persistent reasoning limitations across model scales.

Another line of work constructs rule-based or proof-oriented reasoning tasks to examine whether models can perform multi-step inference over natural-language facts and rules. Representative datasets include RuleTaker [43], ProofWriter [44], EntailmentBank [45], and CLUTRR [46]. These benchmarks provide controlled settings for evaluating deductive reasoning, proof generation, and relational inference. However, they still primarily evaluate model correctness on individual test instances with predefined ground-truth labels.

Prompting strategies have also been widely studied to improve LLM reasoning performance. CoT prompting [47,48], self-consistency decoding based CoT prompting [49], and least-to-most prompting [50] encourage models to generate intermediate reasoning steps or decompose complex problems into simpler subproblems. These methods can improve benchmark accuracy, but they do not directly ensure that model predictions remain invariant under logically equivalent reformulations. This motivates our RQ4, where we examine whether advanced prompting strategies can mitigate the logical defects exposed by LGMT.

Overall, existing reasoning evaluations are still largely centered on static datasets with fixed ground-truth labels. Such evaluations may be affected by data contamination and coincidental correctness [51]. LGMT is complementary to these approaches. Rather than replacing benchmark-based evaluation, it introduces a consistency-based perspective by checking whether LLM predictions remain invariant under logic-preserving transformations. This makes it possible to reveal hidden inconsistencies that are not observable through reference-based evaluation alone.

7.2. Metamorphic testing for NLP and LLMs

MT has long been recognized as an effective mechanism for alleviating the test oracle problem by checking relations among multiple executions [29,32,33,52–54]. Rather than requiring an exact ground-truth label for every individual input, MT verifies whether the outputs of source and follow-up test cases satisfy predefined MRs. This makes MT suitable for testing systems, where manually specifying expected outputs for arbitrary inputs is often expensive or infeasible.

Recently, MT has gained increasing attention in NLP and LLM evaluation [34,55,56]. For example, LLMORPH [34] proposes an automated MT framework for LLMs using a set of lexical MRs. MT has also been adapted to hallucination detection. MetaQA [31] and DrHall [30] employ transformations such as synonym substitution or affirmative-to-negative conversion to expose factual inconsistencies in question answering. Similarly, Drowzee [28] leverages logic programming to construct test scenarios for fact-conflicting hallucination detection.

Despite these advances, many existing MT approaches for NLP still rely on lexical, syntactic, or heuristic MRs. Because natural language is inherently ambiguous, such MRs may introduce unintended semantic drift, which can affect the validity of follow-up test cases [34]. This issue is particularly important for logical reasoning tasks, where small semantic changes may alter the entailment relation.

LGMT differs from these approaches in that its MRs are derived from formal FOL equivalence laws. This design preserves logical semantics at the FOL level, while the natural language translation is explicitly controlled and empirically validated. As a result, LGMT reduces the risk of semantic drift that commonly affects heuristic MT and enables a more controlled consistency-based evaluation of LLM reasoning.

7.3. Neurosymbolic integration for LLMs

Our work is also related to neurosymbolic approaches that combine LLMs with symbolic reasoning systems. Recent frameworks such as Logic-LM [36] and LINC [38] adopt multi-stage pipelines in which LLMs translate natural language into formal representations and external symbolic solvers, such as Prover9 or Z3, perform the reasoning process. Other studies use symbolic tools to support dataset construction, reasoning supervision, or evaluation [25,35,37].

These approaches primarily use formal logic and symbolic solvers to enhance or support LLM reasoning. In contrast, LGMT employs formal logic as an evaluation mechanism rather than an external reasoning component. Instead of bypassing model limitations through symbolic solvers, LGMT uses FOL to construct logic-preserving transformations and then evaluates whether the model itself can maintain consistent judgments across logically equivalent test cases. In this sense, LGMT provides a black-box, consistency-based framework for diagnosing the reasoning reliability of LLMs.

8. Conclusion and future work

In this paper, we proposed LGMT, a logic-grounded metamorphic testing framework for evaluating the logical reasoning reliability of large language models. By deriving metamorphic relations from first-order logic, LGMT generates logic-preserving follow-up test cases and checks whether model predictions remain consistent across source and follow-up cases. This design reduces the semantic drift commonly introduced by lexical perturbations and enables label-free identification of reasoning defects.

Our empirical study shows three main findings. First, LGMT exposes non-trivial reasoning inconsistencies across all evaluated models, while the audited false report rate remains low. Second, compared with traditional reference-based testing, LGMT reveals hidden defects that are not captured by static benchmark accuracy, suggesting that isolated correctness can overestimate reasoning reliability. Third, current LLMs are particularly sensitive to symbol-level and conclusion-level transformations. Although advanced prompting strategies can partially reduce these inconsistencies, they do not eliminate the observed robustness issues in our experiments.

Future work can extend LGMT in several directions. First, *multi-MR validation* can strengthen the current pairwise design by checking consistency across multiple follow-up variants derived from different MRs. Second, *MR composition* can generate more structurally diverse yet logically equivalent follow-up cases, enabling a stronger test of logical consistency. Third, integrating LGMT with lightweight symbolic verification may help reduce the remaining blind spots captured by FUR and provide a more complete assessment of logical robustness.

CRediT authorship contribution statement

Zenghui Zhou: Writing – original draft, Visualization, Software, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Man Li:** Visualization, Software, Investigation, Data curation. **Xiaoke Fang:** Software, Investigation, Data curation. **Xinyi Zhou:** Software, Investigation, Data curation. **Weibin Lin:** Software. **Zheng Zheng:** Writing – review & editing, Funding acquisition, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT to assist with language refinement. The authors reviewed and edited all content and take full responsibility for the final manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was supported by the National Key R & D Program of China under Grant 2024YFB3311503, the National Natural Science Foundation of China under Grant 62372021, the State Key Laboratory of Complex & Critical Software Environment under Grant 606. We would like to thank our friend Jianxing Du for his insightful inspiration and careful verification of the proof presented in the appendix.

Appendix A. Completeness of MR-E1 for PNNF

This appendix formalizes the notion of normalization completeness used in this work and proves it for MR-E1 under the proposed PNNF-oriented rewrite procedure. All discussions and results in this appendix are carried out in first-order logic over the allowed symbol set $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists\}$. Regarding the equality symbol $=$, we treat the atomic formula $t_1 = t_2$ syntactically as a binary predicate $Eq(t_1, t_2)$. Therefore, for the purpose of structural normalization, equality is subsumed under the definition of atomic formulas and is not listed in the allowed symbol set above. And all formulas considered in this appendix are of finite length.

A.1. Syntax and notation

We work in standard first-order logic over a fixed signature. Let Form denote the set of all first-order formulas. For a formula φ, let $\mathrm{FV}(\varphi)$ denote the set of free variables of φ. We write $\varphi[x := y]$ for capture-avoiding substitution of x by y. Two formulas φ and ψ are *α-equivalent*, written $\varphi \equiv_\alpha \psi$, if they differ only by consistent renaming of bound variables. A formula is in *negation normal form (NNF)* if negation occurs only immediately in front of atomic formulas, and the formula contains no implication or biconditional connectives. A formula is in *prenex negation normal form (PNNF)* if it has the form $Q_1 x_1 \cdots Q_k x_k . M$ where each $Q_i \in \{\forall, \exists\}$ and M is a quantifier-free NNF matrix. We define a one-step rewrite relation $\rightsquigarrow$ on formulas induced by the metamorphic relations (MRs). Let $\rightsquigarrow^*$ denote the reflexive transitive closure of $\rightsquigarrow$. Throughout this appendix, equality between formulas is understood up to α-equivalence unless otherwise stated.

A.2. What completeness means in this work

The completeness studied here is *not* semantic completeness of first-order logic. Instead, it is completeness relative to a normalization target, namely PNNF.

Definition 1 (*Completeness w.r.t. PNNF Normalization*)**.** A set of MRs is complete with respect to PNNF normalization if: (i) every first-order formula can be rewritten into a PNNF formula using these MRs, and (ii) the obtained PNNF is unique up to α-equivalence.

In this appendix, we focus on the proof of the completeness of MR-E1.

A.3. The termination of MR-E1

We define a lexicographic measure

$$m(\varphi) = (i(\varphi), n(\varphi), q(\varphi), s(\varphi), v(\varphi), d(\varphi)) \in \mathbb{N}^6.$$

Here $i(\varphi)$ counts the occurrences of implication and biconditional connectives in the formula; $n(\varphi)$ counts occurrences of negation that are not applied directly to atomic formulas; $q(\varphi)$ is the sum of depths of quantifiers that are not in the outermost prenex prefix position; $s(\varphi)$ counts occurrences of conjunctions and disjunctions that are not flattened or not ordered according to the fixed canonical ordering of MR-E1.4; $v(\varphi)$ counts the number of variable conflicts that block the application of MR-E1.3; $d(\varphi)$ counts deviations in the ordering of bound variables within maximal same-quantifier blocks with respect to the canonical order enforced by MR-E1.6. The measure $m(\varphi)$ is compared using the standard lexicographic order on $\mathbb{N}^6$.

Lemma 1 (*Strict Decrease*)**.** *If $\varphi \rightsquigarrow \psi$, then $m(\psi) < m(\varphi)$.*

Proof. We argue by a case analysis on the applied metamorphic relation. Each MR is oriented from non-canonical to canonical form and is designed to strictly decrease one designated component of m.

MR-E1.1 strictly decreases $i(\cdot)$ by eliminating implication and biconditional connectives. MR-E1.2 strictly decreases $n(\cdot)$ by eliminating a non-atomic negation. MR-E1.3 strictly decreases $q(\cdot)$ by lifting a quantifier toward the prenex prefix. MR-E1.4 strictly decreases $s(\cdot)$ by flattening and reordering the formula. MR-E1.6 strictly decreases $d(\cdot)$ by reordering bound variables within the same quantifier block. MR-E1.5 is applicable only when a variable conflict blocks the application of MR-E1.3. Each application of MR-E1.5 eliminates such a conflict and does not introduce new ones, and therefore strictly decreases $v(\cdot)$.

In all cases, the first component in the lexicographic measure (i, n, q, s, v, d) that differs between φ and ψ strictly decreases. Hence $m(\psi) < m(\varphi)$. Since the lexicographic order on $\mathbb{N}^6$ is well founded and therefore cannot decrease infinitely, the rewriting process terminates. Note that a rewrite step may increase some later components of the tuple $m(\cdot)$. This does not affect the lexicographic comparison, because the earliest component at which φ and ψ differ strictly decreases. □

Corollary 1 (*Termination*)**.** *There exists no infinite rewrite sequence under $\rightsquigarrow$.*

A.4. Existence of PNNF normal forms

Lemma 2 (*NNF Reachability*)**.** *For every formula φ, there exists a formula φ_{NNF} such that $\varphi \rightsquigarrow^* \varphi_{\mathrm{NNF}}$ and φ_{NNF} is in NNF.*

Proof. We first repeatedly apply MR-E1.1. Each application of MR-E1.1 strictly decreases $i(\cdot)$, and therefore this process terminates with a formula φ' such that $i(\varphi') = 0$. We then apply repeatedly MR-E1.2 to φ'. Each application of MR-E1.2 strictly decreases $n(\cdot)$, and hence this second process terminates with a formula φ_{NNF} satisfying $n(\varphi_{\mathrm{NNF}}) = 0$. Since $i(\varphi_{\mathrm{NNF}}) = 0$ and $n(\varphi_{\mathrm{NNF}}) = 0$, the resulting formula is in NNF. □

Lemma 3 (*PNNF Reachability from NNF*)**.** *For every NNF formula φ, there exists φ_{PNNF} such that $\varphi \rightsquigarrow^* \varphi_{\mathrm{PNNF}}$ and φ_{PNNF} is in PNNF.*

Proof. Repeatedly apply MR-E1.3. Whenever MR-E1.3 is blocked by its side condition, apply MR-E1.5 to remove the variable conflict. Each MR-E1.3 step strictly decreases $q(\cdot)$. Hence after finitely many steps we obtain a formula φ_{PNNF} with $q(\varphi_{\mathrm{PNNF}}) = 0$. Since φ is in NNF and MR-E1.3/MR-E1.5 do not introduce non-atomic negations, the matrix remains in NNF. Therefore φ_{PNNF} is in PNNF. □

Lemma 4 (*PNNF Reachability*)**.** *For every formula φ, there exists a PNNF formula ψ such that $\varphi \rightsquigarrow^* \psi$.*

Proof. Apply Lemma 2 to obtain an NNF formula. Then apply Lemma 3 to obtain a PNNF formula. □

A.5. Uniqueness of PNNF normal forms

Definition 2 (*Canonical PNNF and* PNNF*)**.** A PNNF formula φ is *canonical* iff $m(\varphi) = (0, 0, 0, 0, 0, 0)$. Equivalently, φ satisfies the canonicality conditions enforced by MR-E1.1–MR-E1.6. We refer to canonical PNNF formulas as PNNF* formulas. We denote the set of all PNNF* formulas by PNNF^*_s.

Definition 3 (*Normalization Procedure NP*)**.** We define a normalization procedure $NP(\cdot)$ that maps any first-order logic formula to a canonical PNNF* formula by a fixed rewrite strategy: First, apply MR-E1.1 and MR-E1.2 exhaustively to reach NNF. Then lift quantifiers by MR-E1.3, using MR-E1.5 when alpha-renaming is needed to satisfy the side condition of quantifier lifting. Next, apply MR-E1.4 to canonicalize the quantifier-free NNF matrix. Finally, apply MR-E1.6 to canonically order each maximal same-quantifier block of the prefix.

Lemma 5 (*Correctness of NP*)**.** *For every formula φ, we have $NP(\varphi) \in \mathrm{PNNF}^*_s$.*

Proof. By Lemma 4, the obtained PNNF is reachable from φ by $\rightsquigarrow^*$ by applying MR-E1.1–MR-E1.3 and MR-E1.5. MR-E1.4 canonicalizes the quantifier-free NNF part of the formula. MR-E1.6 canonically orders bound variables within each maximal same-quantifier block. Therefore $NP(\varphi)$ satisfies all canonicality conditions enforced by MR-E1.1–MR-E1.6. Equivalently, we have $m(NP(\varphi)) = (0, 0, 0, 0, 0, 0)$. Hence $NP(\varphi) \in \mathrm{PNNF}^*_s$. □

Lemma 6 (*Self-Consistency of NP*)**.** *For every formula φ, the result $NP(\varphi)$ is unique up to α-equivalence.*

Proof. By construction, $NP(\varphi) \in \mathrm{PNNF}^*_s$. By applying $NP(\cdot)$ to φ, the syntactic structure of $NP(\varphi)$ is fixed by the canonicality conditions induced by MR-E1.1–MR-E1.6. Consequently, any two results obtained by applying NP to φ can differ only by α-renaming of bound variables, and are therefore α-equivalent. □

Lemma 7 (*Normalization Invariance under $\rightsquigarrow$*)**.** *If $\varphi \rightsquigarrow \psi$, then $NP(\varphi) \equiv_\alpha NP(\psi)$.*

Proof. A rewrite step $\varphi \rightsquigarrow \psi$ is an application of a single metamorphic relation and thus represents an alternative syntactic presentation of the same formula within the normalization system. By Lemma 5, the normalization procedure NP terminates on any input and produces a canonical normal form that is unique up to α-equivalence. Therefore, applying NP to φ and to ψ yields α-equivalent results. Hence $NP(\varphi) \equiv_\alpha NP(\psi)$. □

Lemma 8 (*Normalization Invariance under $\rightsquigarrow^*$*)**.** *If $\varphi \rightsquigarrow^* \psi$, then $NP(\varphi) \equiv_\alpha NP(\psi)$.*

Proof. We prove the claim by induction on the length of a rewrite sequence witnessing $\varphi \rightsquigarrow^* \psi$. If the sequence has length 0, then $\psi = \varphi$. By Lemma 6, we have $NP(\varphi) \equiv_\alpha NP(\psi)$. Otherwise, there exists a formula θ such that $\varphi \rightsquigarrow^* \theta$ and $\theta \rightsquigarrow \psi$. By the induction hypothesis, $NP(\varphi) \equiv_\alpha NP(\theta)$. By Lemma 7, $NP(\theta) \equiv_\alpha NP(\psi)$. By transitivity of $\equiv_\alpha$, we obtain $NP(\varphi) \equiv_\alpha NP(\psi)$. □

Lemma 9 (*Fixpoint Property*). *If ψ is a* PNNF* *formula, then $NP(\psi) \equiv_\alpha \psi$.*

Proof. Since $\psi \in$ PNNF*, we have $m(\psi) = (0,0,0,0,0,0)$. Hence no rewrite step of MR-E1.1–MR-E1.6 is applicable that would further decrease m. Therefore NP does not modify the syntactic structure of ψ. Hence $NP(\psi) \equiv_\alpha \psi$. □

Theorem 1 (*PNNF-Completeness*). *For every formula φ, there exists a unique (up to α-equivalence)* PNNF* *formula ψ such that $\varphi \rightsquigarrow^* \psi$.*

Proof. We first prove existence. Let $\psi := NP(\varphi)$. By construction, $NP(\varphi) \in$ PNNF*. Since NP is defined as a rewrite strategy using MR-E1.1–MR-E1.6, we have $\varphi \rightsquigarrow^* NP(\varphi)$. Therefore $\varphi \rightsquigarrow^* \psi$ with $\psi \in \text{PNNF}^*_s$.

We next prove uniqueness. Assume $\varphi \rightsquigarrow^* \psi_1$ and $\varphi \rightsquigarrow^* \psi_2$ with $\psi_1, \psi_2 \in$ PNNF*. By Lemma 8, we have $NP(\varphi) \equiv_\alpha NP(\psi_1)$ and $NP(\varphi) \equiv_\alpha NP(\psi_2)$. By Lemma 9, we have $NP(\psi_1) \equiv_\alpha \psi_1$ and $NP(\psi_2) \equiv_\alpha \psi_2$. Hence $NP(\varphi) \equiv_\alpha \psi_1$ and $NP(\varphi) \equiv_\alpha \psi_2$. By transitivity of $\equiv_\alpha$, we conclude $\psi_1 \equiv_\alpha \psi_2$. □

Appendix B. Illustrative end-to-end example of LGMT

We provide a concrete example to illustrate the full LGMT pipeline, including source test case construction, logic-grounded transformation, and defect detection.

(1) source test case. The original reasoning instance is shown below.

Premises

1. Lawton Park is a neighborhood in Seattle.
2. All citizens of Lawton Park use the zip code 98199.
3. Tom is a citizen of Lawton Park.
4. Daniel uses the zip code 98199.

Conclusion

Tom is a citizen of Washington.

The ground-truth label of this instance is **Unknown**, since no premise connects Lawton Park or Seattle to Washington.

(2) FOL representation. The corresponding symbolic representation is shown below.

Premises

1. `NeighbourhoodIn(lawtonPark, seattle)`
2. `forall x. (ResidentOf(x, lawtonPark) -> UseZipCode(x, num98199))`
3. `ResidentOf(tom, lawtonPark)`
4. `UseZipCode(daniel, num98199)`

Conclusion

`ResidentOf(tom, washington)`

(3) applied metamorphic relation. We apply MR-E1.1 (implication elimination), which rewrites an implication into a logically equivalent disjunction:

$$\forall x.\ (P(x) \to Q(x)) \rightsquigarrow \forall x.\ (\neg P(x) \vee Q(x)).$$

After transformation, the second premise becomes:

`forall x. (not ResidentOf(x, lawtonPark) or UseZipCode(x, num98199))`

(4) generated follow-up test case. The transformed FOL formula is translated back into natural language, yielding the follow-up test case shown below.

Follow-up Premises

1. Lawton Park is a neighborhood in Seattle.
2. For every person, either they are not a citizen of Lawton Park, or they use the zip code 98199.
3. Tom is a citizen of Lawton Park.
4. Daniel uses the zip code 98199.

Conclusion

Tom is a citizen of Washington.

(5) model outputs and oracle decision. Let the model outputs for the source and follow-up test cases be denoted as y_s and y_f, respectively. Under LGMT, a *metamorphic oracle violation* occurs if

$$y_s \neq y_f$$

Since the transformation preserves logical equivalence, the correct reasoning outcome should remain unchanged. Any inconsistency between y_s and y_f therefore indicates a *logical reasoning defect*.

Appendix C. Fine-grained sensitivity analysis

As shown in Table C.10, the fine-grained results are broadly consistent with the category-level trends reported in Section 5.3. In particular, several MRs under MR-S and MR-C exhibit relatively high violation rates, such as MR-S2 (34.03%) and MR-C2 (37.50%), further confirming that symbol-level and conclusion-level transformations are especially challenging for current LLMs. By contrast, most premise-level transformations remain comparatively less disruptive, with MR-P1, MR-P3, and MR-P5 all below 15%. We also observe substantial heterogeneity within MR-E: while some equivalence transformations (e.g., MR-E2.3) do not have results due to the absence of applicable MG instances in our dataset, others induce much higher inconsistency (e.g., MR-E1.5 with 35.61%), suggesting that the difficulty of formula-level transformations depends strongly on the specific rewrite type.

C.1. Representative MR-S2 failure case

We provide a concrete MR-S2 failure case to illustrate why symbol-level transformations can expose reasoning instability.

Table C.10
Sensitivity of LGMT across MRs. Each cell reports the Metamorphic Violation Rate (MVR). The Avg. MVR column is computed by aggregating violations across all models.

Category	Avg. MVR	GPT-5.2	Claude 4.5 Sonnet	Llama 3.3	DeepSeek Chat	o4-mini	DeepSeek reasoner
MR-E1.1	26.50%	27.00%	26.00%	27.00%	25.50%	25.00%	28.50%
MR-E1.2	32.25%	34.00%	27.50%	32.00%	37.00%	31.00%	32.00%
MR-E1.3	33.33%	21.43%	26.19%	33.33%	35.71%	38.10%	45.24%
MR-E1.4	18.92%	22.50%	12.50%	17.00%	17.00%	16.50%	28.00%
MR-E1.5	35.61%	40.91%	31.82%	40.91%	36.36%	31.82%	31.82%
MR-E1.6	15.58%	16.50%	10.50%	12.50%	16.00%	17.00%	21.00%
MR-E2.1	16.67%	25.00%	25.00%	16.67%	8.33%	8.33%	16.67%
MR-E2.2	6.67%	0.00%	0.00%	13.33%	6.67%	13.33%	6.67%
MR-E2.3	–	–	–	–	–	–	–
MR-E2.4	29.67%	26.00%	29.50%	34.50%	28.00%	32.00%	28.00%
MR-S1	32.08%	29.50%	28.50%	34.00%	35.50%	30.50%	34.50%
MR-S2	34.03%	34.50%	35.18%	36.50%	34.50%	25.00%	38.50%
MR-P1	14.83%	20.00%	8.50%	16.50%	10.00%	13.00%	21.00%
MR-P2	15.92%	17.00%	12.50%	12.50%	19.50%	14.00%	20.00%
MR-P3	14.83%	18.50%	8.50%	16.00%	13.00%	11.50%	21.50%
MR-P4	22.17%	21.50%	17.50%	22.00%	24.50%	18.00%	29.50%
MR-P5	13.92%	18.50%	6.00%	13.50%	12.50%	15.50%	17.50%
MR-C1	30.42%	44.00%	25.50%	22.50%	25.50%	27.50%	37.50%
MR-C2	37.50%	39.50%	34.00%	35.50%	39.50%	23.50%	53.00%
MR-C3	33.42%	34.00%	29.50%	47.50%	31.50%	28.00%	30.00%

(1) source test case. The original reasoning instance is shown below.

Premises

1. Machine Learning algorithms can be categorized as supervised learning, unsupervised learning, and reinforcement learning.

2. Unsupervised learning algorithms do not require labeled data.

3. The state-of-the-art text summarization model is trained with machine learning algorithms.

4. Reinforcement learning is not used to train the state-of-the-art text summarization model.

5. The Machine Learning algorithm for training text summarization models requires labeled data.

Conclusion

Supervised learning is used to train the state-of-the-art text summarization model.

The expected label is **True**. The tested LLM also predicts *True* for this source case.

(2) applied metamorphic relation. We apply MR-S2 (predicate renaming invariance), which uniformly replaces non-logical predicates with fresh abstract symbols while preserving the underlying logical structure.

(3) generated follow-up test case. After predicate renaming, the follow-up test case is shown below.

Follow-up Premises

1. For all x, if x has property $Pre1$, then either x has property $Pre4$, or x has property $Pre6$, or x has property $Pre2$.

2. For all x, if x has property $Pre6$, then it is not the case that x bears relation $Pre3$ to labeledData.

3. For all x, if the state-of-the-art text summarization model bears relation $Pre5$ to x, then x has property $Pre1$.

4. For all x, if x has property $Pre2$, then it is not the case that stateOfTheArtTextSummarizationModel bears relation $Pre5$ to x.

5. For all x, if both x has property $Pre1$ and stateOfTheArtTextSummarizationModel bears relation $Pre5$ to x, then x bears relation $Pre3$ to labeledData.

Conclusion

There exists at least one x, such that both x has property $Pre4$, and stateOfTheArtTextSummarizationModel bears relation $Pre5$ to x.

(4) model outputs and oracle decision. For the source test case, the model predicts *True*. For the follow-up test case, however, it predicts *Unknown*, arguing that the premises do not guarantee the required existential conclusion. Since MR-S2 preserves the logical structure, the expected output relation is:

$$y_s = y_f$$

The observed prediction change therefore violates the metamorphic oracle and indicates sensitivity to predicate names rather than purely abstract logical structure.

Data availability

Artifacts (code, prompts, configurations, and processed data) are available at an anonymized repository https://anonymous.4open.science/r/LGMT_code-8A0D/README.md.

References

[1] DeepSeek, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, et al., DeepSeek-V3 technical report, 2025, http://dx.doi.org/10.48550/arXiv.2412.19437.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474.

[3] M. Yue, A survey of large language model agents for question answering, 2025, http://dx.doi.org/10.48550/arXiv.2503.19213.

[4] Y. Zhuang, Y. Yu, K. Wang, H. Sun, C. Zhang, ToolQA: A dataset for LLM question answering with external tools, Adv. Neural Inf. Process. Syst. 36 (2023) 50117–50143.

[5] DeepSeek-AI, Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, et al., DeepSeek-coder-V2: Breaking the barrier of closed-source models in code intelligence, 2024, http://dx.doi.org/10.48550/arXiv.2406.11931.

[6] C.E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, et al., SWE-bench: Can language models resolve real-world GitHub issues?, 2024, http://dx.doi.org/10.48550/arXiv.2310.06770.

[7] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, et al., Code llama: Open foundation models for code, 2024, http://dx.doi.org/10.48550/arXiv.2308.12950.

[8] Y. Guan, D. Wang, Z. Chu, S. Wang, F. Ni, R. Song, et al., Intelligent virtual assistants with LLM-based process automation, 2023, URL: https://arxiv.org/abs/2312.06677, arXiv:2312.06677.

[9] S. Park, H. Subramonyam, C. Kulkarni, Thinking assistants: LLM-based conversational assistants that help users think by asking rather than answering, 2024, URL: https://arxiv.org/abs/2312.06024, arXiv:2312.06024.

[10] W.H. Holliday, M. Mandelkern, C.E. Zhang, Conditional and modal reasoning in large language models, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2024, pp. 3800–3821, http://dx.doi.org/10.18653/v1/2024.emnlp-main.222.

[11] M. Luo, S. Kumbhar, M. shen, M. Parmar, N. Varshney, P. Banerjee, et al., Towards LogiGLUE: A brief survey and a benchmark for analyzing logical reasoning capabilities of language models, 2023, http://dx.doi.org/10.48550/ARXIV.2310.00836.

[12] S. Singh, Are large language models good at fuzzy reasoning? in: Proceedings of the International Conference on Computational Intelligence and Intelligent Systems, 2024, pp. 1–6, http://dx.doi.org/10.1145/3708778.3708779.

[13] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, et al., A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Trans. Inf. Syst. 43 (2) (2025) 1–55, http://dx.doi.org/10.1145/3703155.

[14] C. Woesle, L. Fischer-Brandies, R. Buettner, A systematic literature review of hallucinations in large language models, IEEE Access 13 (2025) 148231–148253, http://dx.doi.org/10.1109/ACCESS.2025.3601206.

[15] F. Xu, Q. Lin, J. Han, T. Zhao, J. Liu, E. Cambria, Are large language models really good logical reasoners? A comprehensive evaluation and beyond, IEEE Trans. Knowl. Data Eng. 37 (4) (2025) 1620–1634, http://dx.doi.org/10.1109/TKDE.2025.3536008.

[16] Y. Wan, W. Wang, Y. Yang, Y. Yuan, J.t. Huang, P. He, et al., LogicAsker: Evaluating and improving the logical reasoning ability of large language models, in: Proceedings of the International Conference on Empirical Methods in Natural Language Processing, 2024, pp. 2124–2155, http://dx.doi.org/10.18653/v1/2024.emnlp-main.128.

[17] B. Ghosh, S. Hasan, N.A. Arafat, A. Khan, Logical consistency of large language models in fact-checking, in: The Thirteenth International Conference on Learning Representations, 2025, URL: https://openreview.net/forum?id=SimlDuN0YT.

[18] H. Liu, Y. Ding, Z. Fu, C. Zhang, X. Liu, Y. Zhang, Evaluating the logical reasoning abilities of large reasoning models, 2025, http://dx.doi.org/10.48550/arXiv.2505.11854.

[19] W. Yu, Z. Jiang, Y. Dong, J. Feng, Reclor: a reading comprehension dataset requiring logical reasoning, in: Proceedings of the International Conference on Learning Representations, 2020, http://dx.doi.org/10.48550/arXiv.2002.04326.

[20] D. Zhang, Z.Z. Li, M.L. Zhang, J. Zhang, Z. Liu, Y. Yao, et al., From system 1 to system 2: A survey of reasoning large language models, IEEE Trans. Pattern Anal. Mach. Intell. (2025) 1–20, http://dx.doi.org/10.1109/TPAMI.2025.3637037.

[21] S. Han, H. Schoelkopf, Y. Zhao, Z. Qi, M. Riddell, W. Zhou, et al., FOLIO: natural language reasoning with first-order logic, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2024, pp. 22017–22031, http://dx.doi.org/10.18653/v1/2024.emnlp-main.1229.

[22] M. Parmar, N. Patel, N. Varshney, M. Nakamura, M. Luo, S. Mashetty, et al., LogicBench: Towards systematic evaluation of logical reasoning ability of large language models, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2024, pp. 13679–13707, http://dx.doi.org/10.18653/v1/2024.acl-long.739.

[23] J. Liu, L. Cui, H. Liu, D. Huang, Y. Wang, Y. Zhang, Logiqa: a challenge dataset for machine reading comprehension with logical reasoning, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2021, pp. 3622–3628, URL: https://dl.acm.org/doi/10.5555/3491440.3491941.

[24] R. Xu, Z. Wang, R.Z. Fan, P. Liu, Benchmarking benchmark leakage in large language models, 2024, http://dx.doi.org/10.48550/arXiv.2404.18824.

[25] C. Qi, R. Ma, B. Li, H. Du, B. Hui, J. Wu, et al., Large language models meet symbolic provers for logical reasoning evaluation, in: Proceeding of the International Conference on Learning Representations, 2024, URL: https://openreview.net/forum?id=C25SgeXWjE.

[26] N. Dziri, X. Lu, M. Sclar, X.L. Li, L. Jiang, B.Y. Lin, et al., Faith and fate: limits of transformers on compositionality, in: Proceedings of the International Conference on Neural Information Processing Systems, 2023, pp. 70293–70332.

[27] A. Saparov, H. He, Language models are greedy reasoners: a systematic formal analysis of chain-of-thought, in: The International Conference on Learning Representations, 2022, URL: https://openreview.net/forum?id=qFVVBzXxR2V.

[28] N. Li, Y. Li, Y. Liu, L. Shi, K. Wang, H. Wang, Drowzee: metamorphic testing for fact-conflicting hallucination detection in large language models, Proc. ACM Program. Lang. 8 (2024) 1843–1872, http://dx.doi.org/10.1145/3689776.

[29] T.Y. Chen, F.C. Kuo, H. Liu, P.L. Poon, D. Towey, T.H. Tse, et al., Metamorphic testing: a review of challenges and opportunities, ACM Comput. Surv. 51 (1) (2019) 1–27, http://dx.doi.org/10.1145/3143561.

[30] W. Wu, Y. Cao, N. Yi, R. Ou, Z. Zheng, Detecting and reducing the factual hallucinations of large language models with metamorphic testing, Proc. ACM Softw. Eng. 2 (2025) 1432–1453, http://dx.doi.org/10.1145/3715784.

[31] B. Yang, M.A. Al Mamun, J.M. Zhang, G. Uddin, Hallucination detection in large language models with metamorphic relations, Proc. ACM Softw. Eng. 2 (2025) 425–445, http://dx.doi.org/10.1145/3715735.

[32] T.Y. Chen, S.C. Cheung, S.M. Yiu, Metamorphic testing: A new approach for generating next test cases, Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998, p. 11.

[33] S. Chen, S. Jin, X. Xie, Testing your question answering software via asking recursively, in: Preceeding of the International Conference on Automated Software Engineering, 2021, pp. 104–116, http://dx.doi.org/10.1109/ASE51524.2021.9678670.

[34] S. Cho, S. Ruberto, V. Terragni, Metamorphic testing of large language models for natural language processing, 2025, http://dx.doi.org/10.48550/arXiv.2511.02108.

[35] C. Cao, M. Li, J. Dai, J. Yang, Z. Zhao, S. Zhang, et al., Towards advanced mathematical reasoning for LLMs via first-order logic theorem proving, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2025, pp. 12429–12449, http://dx.doi.org/10.18653/v1/2025.emnlp-main.628.

[36] L. Pan, A. Albalak, X. Wang, W.Y. Wang, Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning, in: Proceeding of the Conference on Empirical Methods in Natural Language Processing, 2023, URL: https://openreview.net/forum?id=nWXMv949ZH&noteId=qt0t8SsVvT.

[37] J. Jiang, J. Wang, Y. Yan, Y. Liu, J. Zhu, M. Zhang, et al., Do large language models excel in complex logical reasoning with formal language? in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 16889–16914, http://dx.doi.org/10.18653/v1/2025.emnlp-main.855.

[38] T. Olausson, A. Gu, B. Lipkin, C. Zhang, A. Solar-Lezama, J. Tenenbaum, et al., LINC: a neurosymbolic approach for logical reasoning by combining language models with first-order logic provers, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2023, pp. 5153–5176, http://dx.doi.org/10.18653/v1/2023.emnlp-main.313.

[39] J. Tian, Y. Li, W. Chen, L. Xiao, H. He, Y. Jin, Diagnosing the first-order logical reasoning ability through logicnli, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2021, pp. 3738–3747, http://dx.doi.org/10.18653/v1/2021.emnlp-main.303.

[40] W.G. Cochran, Errors of measurement in statistics, Technometrics 10 (4) (1968) 637–666, http://dx.doi.org/10.1080/00401706.1968.10490621.

[41] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, et al., Think you have solved question answering? Try ARC, the AI2 reasoning challenge, 2018, http://dx.doi.org/10.48550/ARXIV.1803.05457.

[42] N. Patel, M. Kulkarni, M. Parmar, A. Budhiraja, M. Nakamura, N. Varshney, et al., Multi-LogiEval: towards evaluating multi-step logical reasoning ability of large language models, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2024, pp. 20856–20879, http://dx.doi.org/10.18653/v1/2024.emnlp-main.1160.

[43] P. Clark, O. Tafjord, K. Richardson, Transformers as soft reasoners over language, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2021, pp. 3882–3890.

[44] O. Tafjord, B. Dalvi, P. Clark, ProofWriter: generating implications, proofs, and abductive statements over natural language, in: Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, 2021, pp. 3621–3634, http://dx.doi.org/10.18653/v1/2021.findings-acl.317.

[45] B. Dalvi, P. Jansen, O. Tafjord, Z. Xie, H. Smith, L. Pipatanangkura, et al., Explaining answers with entailment trees, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2021, pp. 7358–7370, http://dx.doi.org/10.18653/v1/2021.emnlp-main.585.

[46] K. Sinha, S. Sodhani, J. Dong, J. Pineau, W.L. Hamilton, CLUTRR: a diagnostic benchmark for inductive reasoning from text, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing, 2019, pp. 4505–4514, http://dx.doi.org/10.18653/v1/D19-1458.

[47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, et al., Chain-of-thought prompting elicits reasoning in large language models, in: Proceedings of the International Conference on Neural Information Processing Systems, 2022, pp. 24824–24837.

[48] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H.W. Chung, et al., Challenging BIG-bench tasks and whether chain-of-thought can solve them, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 13003–13051, http://dx.doi.org/10.18653/v1/2023.findings-acl.824.

[49] X. Wang, J. Wei, D. Schuurmans, Q.V. Le, E.H. Chi, S. Narang, et al., Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations, 2022.

[50] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, et al., Least-to-most prompting enables complex reasoning in large language models, 2023, http://dx.doi.org/10.48550/arXiv.2205.10625.

[51] P. Mondorf, B. Plank, Beyond accuracy: Evaluating the reasoning behavior of large language models - A survey, in: First Conference on Language Modeling, 2024, URL: https://openreview.net/forum?id=Lmjgl2n11u.

[52] S. Segura, G. Fraser, A.B. Sanchez, A. Ruiz-Cortés, A survey on metamorphic testing, IEEE Trans. Softw. Eng. 42 (9) (2016) 805–824, http://dx.doi.org/10.1109/TSE.2016.2532875.

[53] C. Murphy, G.E. Kaiser, L. Hu, L. Wu, Properties of machine learning applications for use in metamorphic testing, in: Proceedings of the International Conference on Software Engineering & Knowledge Engineering, Knowledge Systems Institute Graduate School, 2008, pp. 867–872.

[54] X. Xie, J.W.K. Ho, C. Murphy, G. Kaiser, B. Xu, T.Y. Chen, Testing and validating machine learning classifiers by metamorphic testing, J. Syst. Softw. 84 (4) (2011) 544–558, http://dx.doi.org/10.1016/j.jss.2010.11.920.

[55] C. Sok, D. Luz, Y. Haddam, MetaRAG: Metamorphic testing for hallucination detection in RAG systems, 2025, http://dx.doi.org/10.48550/arXiv.2509.09360.

[56] B. Yang, Y. Xia, W. Sun, Y. Liu, Hallucination detection for LLM-based text-to-SQL generation via two-stage metamorphic testing, 2025, http://dx.doi.org/10.48550/arXiv.2512.22250.